TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



Trần Hồ Duy Ân - 52400065
Nguyễn Anh Quân - 52400310

BÁO CÁO GIỮA KỲ

LẬP TRÌNH WEB VÀ ỨNG DỤNG

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



Trần Hồ Duy Ân - 52400065
Nguyễn Anh Quân - 52400310

BÁO CÁO GIỮA KỲ

LẬP TRÌNH WEB VÀ ỨNG DỤNG

Người hướng dẫn
ThS. Nguyễn Thái Duy

Commented [PC1]: Ghi đúng học hàm (nếu có), học vị, họ tên giảng viên phụ trách lớp học
GS: Giáo sư
PGS. TS.: Phó giáo sư.Tiến sĩ
TS: Tiến sĩ
ThS: Thạc sĩ

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

LỜI CẢM ƠN

Chúng em xin chân thành cảm ơn thầy ThS. Nguyễn Thái Duy đã tận tình giảng dạy, định hướng chuyên môn và hỗ trợ chúng em trong suốt quá trình học tập cũng như thực hiện báo cáo giữa kỳ môn Lập trình web và ứng dụng (mã môn học: 503073). Những kiến thức nền tảng về kiến trúc dịch vụ Web, giao thức truyền thông HTTP, cơ chế bảo mật và phương pháp thiết kế API mà thầy truyền đạt đã giúp chúng em xây dựng thành công hệ thống thực nghiệm hoàn chỉnh. Bên cạnh đó, chúng em cũng xin gửi lời cảm ơn chân thành đến quý thầy cô Khoa Công nghệ thông tin, Trường Đại học Tôn Đức Thắng đã tạo điều kiện học tập thuận lợi cho chúng em. Dù đã nỗ lực hoàn thiện mã nguồn và kiểm thử nghiêm túc, báo cáo khó tránh khỏi những thiếu sót nhất định. Chúng em rất mong nhận được những góp ý quý báu từ thầy để tiếp tục hoàn thiện kỹ năng phát triển phần mềm trong tương lai.

TP. Hồ Chí Minh, ngày 01 tháng 11 năm 2026

Tác giả

(Ký tên và ghi rõ họ tên)

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH
TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Chúng em xin cam đoan đây là công trình nghiên cứu và phát triển phần mềm của nhóm chúng em dưới sự hướng dẫn khoa học của ThS. Nguyễn Thái Duy. Toàn bộ thiết kế kiến trúc, mã nguồn ứng dụng và kết quả thực nghiệm trình bày trong đề tài này là trung thực, do chính nhóm tự lập trình và kiểm thử.

Các số liệu, bảng biểu, sơ đồ và tài liệu tham khảo trong báo cáo đều có nguồn gốc rõ ràng và trích dẫn theo đúng quy định.

Nếu phát hiện có bất kỳ sự gian lận nào, chúng em xin hoàn toàn chịu trách nhiệm về nội dung Dự án của nhóm mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền do chúng em gây ra trong quá trình thực hiện.

TP. Hồ Chí Minh, ngày 01 tháng 11 năm 2026
Tác giả
(Ký tên và ghi rõ họ tên)

NGHIÊN CỨU KIẾN TRÚC DỊCH VỤ WEB RESTFUL VÀ XÂY DỰNG HỆ THỐNG QUẢN LÝ THƯ VIỆN MƯỢN – TRẢ SÁCH (READOWL LIBRARY)

TÓM TẮT

Báo cáo này nghiên cứu kiến trúc Dịch vụ Web RESTful (Representational State Transfer) và ứng dụng trực tiếp vào việc xây dựng hệ thống quản lý thư viện mượn – trả sách trực tuyến ReadOwl Library. Hệ thống được phát triển theo mô hình tách biệt hoàn toàn giữa giao diện người dùng và dịch vụ dữ liệu phía máy chủ. Tầng dịch vụ RESTful API được xây dựng bằng ngôn ngữ PHP thuần (PHP 8.2), cung cấp 25 điểm cuối (endpoints) bám sát các phương thức chuẩn của giao thức HTTP (GET, POST, PUT, DELETE) và trả về định dạng dữ liệu chuẩn JSON. Hệ thống áp dụng cơ chế xác thực phi trạng thái (Stateless) bằng JSON Web Token (JWT) theo thuật toán HMAC-SHA256 kết hợp phân quyền người dùng (Role-Based Access Control – RBAC). Cơ sở dữ liệu quan hệ MySQL sử dụng động cơ lưu trữ InnoDB với cơ chế giao dịch nguyên tử (ACID Transaction) và khóa dòng dữ liệu (SELECT ... FOR UPDATE) nhằm đảm bảo tính toàn vẹn tuyệt đối của số lượng sách tồn kho khi xảy ra mượn trả đồng thời. Giao diện người dùng được xây dựng bằng HTML5, CSS3 và JavaScript Fetch API bất đồng bộ. Hợp đồng giao tiếp được tài liệu hóa bằng đặc tả OpenAPI 3.0.3 và kiểm chứng tự động thông qua 65 ca kiểm thử tích hợp trên giao diện dòng lệnh (CLI) cùng bộ sưu tập kiểm thử Postman.

A STUDY ON RESTFUL WEB SERVICES ARCHITECTURE AND DEVELOPMENT OF THE READOWL LIBRARY MANAGEMENT SYSTEM

ABSTRACT

This report investigates the architecture of RESTful Web Services and its direct application in the design and development of the ReadOwl Library online book circulation management system. The system adopts a completely decoupled architecture separating the client-side presentation layer from the server-side API backend. The RESTful API layer is developed using native PHP 8.2, exposing 25 standardized endpoints operating over standard HTTP verbs (GET, POST, PUT, DELETE) with uniform JSON payload envelopes. Stateless authentication is implemented via JSON Web Tokens (JWT) signed with HMAC-SHA256, combined with Role-Based Access Control (RBAC). The underlying MySQL relational database runs on the InnoDB engine, employing ACID transactions and row-level locking (SELECT ... FOR UPDATE) to ensure strict inventory consistency under concurrent borrowing requests. The client application is built using HTML5, CSS3, and modern asynchronous JavaScript Fetch API. The entire API contract is documented using OpenAPI 3.0.3 and thoroughly verified through 65 automated command-line integration tests and Postman test suites.

MỤC LỤC

DANH MỤC HÌNH VẼ viii

DANH MỤC BẢNG BIỂU x

DANH MỤC CÁC CHỮ VIẾT TẮT xii

CHƯƠNG 1. MỞ ĐẦU VÀ TỔNG QUAN ĐỀ TÀI 1

1.1 Lý do chọn đề tài 1

1.2 Mục tiêu thực hiện đề tài 2

1.2.1 Mục tiêu nghiên cứu lý thuyết và phát triển thực nghiệm 2

1.2.2 Phạm vi thực hiện và đối tượng sử dụng hệ thống 2

1.2.3 Bố cục của báo cáo 2

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 4

2.1 Lịch sử hình thành và nguyên lý cốt lõi của kiến trúc REST 4

2.1.1 Bối cảnh ra đời của dịch vụ Web và luận án của Roy Fielding 4

2.1.2 Bốn ràng buộc kiến trúc cốt lõi áp dụng trong hệ thống 4

2.1.3 Tài nguyên, biểu diễn tài nguyên và mô hình trưởng thành Richardson 5

2.2 Đặc tả kỹ thuật giao thức HTTP và thực tiễn thiết kế RESTful API 6

2.2.1 Ngữ nghĩa phương thức HTTP, ánh xạ CRUD, tính an toàn và tính lũy đẳng 6

2.2.2 Cấu trúc thông điệp HTTP và hệ thống mã trạng thái chuẩn 7

2.2.3 Thực tiễn thiết kế URI, phân trang và chuẩn hóa hợp đồng OpenAPI 3.0.3 ... 8

2.3 Cơ chế bảo mật và xác thực trong dịch vụ Web RESTful 9

2.3.1 Cấu trúc JSON Web Token (JWT) và cơ chế xác thực chữ ký số HMAC-SHA256 9

2.3.2 So sánh cơ chế xác thực Session-Cookie và Bearer Token phi trạng thái 9

2.3.3 Các mối đe dọa bảo mật API phổ biến theo OWASP và chiến lược phòng vệ 10

2.4 Phân tích ưu nhược điểm, thách thức kỹ thuật và so sánh các kiến trúc dịch vụ Web 11

2.4.1 Điểm mạnh, điểm yếu và giải pháp giảm thiểu trong dự án..... 11

2.4.2 So sánh tổng quan giữa RESTful API, GraphQL và gRPC 11

CHƯƠNG 3. MÔ HÌNH ĐỀ XUẤT 13

3.1 Tổng quan hệ thống ReadOwl Library và phân tích yêu cầu 13

3.1.1 Mục đích hệ thống, đối tượng người dùng mục tiêu và các tính năng cốt lõi 13

3.1.2 Đặc tả yêu cầu chức năng và yêu cầu phi chức năng..... 13

3.1.3 Mô hình hóa ca sử dụng (Use Case Modeling) 15

3.2 Kiến trúc hệ thống và lựa chọn ngăn xếp công nghệ 15

3.2.1 Kiến trúc tổng thể bốn tầng tách biệt và nguyên lý phối hợp các thành phần 15

3.2.2 Lựa chọn ngăn xếp công nghệ và lý giải kiến trúc 17

3.2.3 Sơ đồ tuần tự các luồng nghiệp vụ trọng tâm..... 17

3.3 Thiết kế cơ sở dữ liệu, hợp đồng REST API và áp dụng lý thuyết vào thực nghiệm..... 18

3.3.1 Thiết kế lược đồ cơ sở dữ liệu quan hệ và các ràng buộc toàn vẹn 18

3.3.2 Thiết kế không gian tài nguyên URI, máy trạng thái phiếu mượn và hợp đồng JSON 19

3.3.3 Áp dụng các khái niệm lý thuyết Chương 2 vào hiện thực mã nguồn 21

CHƯƠNG 4. THỰC NGHIỆM 25

4.1 Dữ liệu thực nghiệm.....25

4.1.1 Cấu trúc bộ dữ liệu khởi tạo (Seed Dataset) và tài khoản kiểm thử.....25

4.1.2 Thiết kế ma trận kịch bản kiểm thử tích hợp tự động (Automated API Test Suites) 27

4.2 Cài đặt thực nghiệm 29

4.2.1 Môi trường phần cứng, phần mềm và cấu trúc cây thư mục dự án.....29

4.2.2 Quy trình triển khai hệ thống và khởi chạy dịch vụ thực nghiệm.....29

4.3 Kết quả và đánh giá.....30

4.3.1 Kết quả thực nghiệm giao diện ứng dụng Web tích hợp RESTful API.....30

4.3.2 Kết quả kiểm thử tự động, kiểm thử Postman và đánh giá hiệu năng.....33

4.4 Thách thức gặp phải và cách giải quyết.....35

4.4.1 Các thách thức kỹ thuật phát sinh trong quá trình phát triển và tích hợp 35

4.4.2 Giải pháp kỹ thuật khắc phục và bài học kinh nghiệm thực tiễn.....35

CHƯƠNG 5. KẾT LUẬN.....38

5.1 Kết luận 38

5.2 Hướng phát triển 38

TÀI LIỆU THAM KHẢO 40

PHỤ LỤC 42

DANH MỤC HÌNH VẼ

Hình 1.1: Sự chuyển dịch từ kiến trúc Web nguyên khối sang kiến trúc RESTful Client–Server tách biệt 1

Hình 2.1: Mô hình trưởng thành Richardson (Richardson Maturity Model) và vị trí hiện thực của dự án ReadOwl Library 5

Hình 2.2: Cấu trúc ba thành phần của JSON Web Token và quy trình xác thực Bearer Token phi trạng thái 9

Hình 3.1: Sơ đồ Use Case tổng thể của hệ thống quản lý thư viện ReadOwl Library 15

Hình 3.2: Sơ đồ kiến trúc tổng thể 4 tầng của hệ thống ReadOwl Library và luồng xử lý yêu cầu RESTful API 16

Hình 3.3: Sơ đồ tuần tự (Sequence Diagram) luồng đăng nhập cấp phát JWT và xác thực yêu cầu truy cập tài nguyên bảo vệ 18

Hình 3.4: Sơ đồ tuần tự (Sequence Diagram) luồng tạo phiếu mượn sách với giao dịch nguyên tử (ACID Transaction) và khóa dòng dữ liệu 18

Hình 3.5: Sơ đồ quan hệ thực thể (ERD) của cơ sở dữ liệu library_rest trong hệ thống ReadOwl Library 19

Hình 3.6: Sơ đồ máy trạng thái hữu hạn (Finite State Machine) của tài nguyên Phiếu mượn (Loan) và cơ chế đồng bộ tồn kho sách 20

Hình 4.1: Giao diện dòng lệnh thực thi kịch bản khởi tạo xác thực (scripts/init-auth.php) và khởi chạy máy chủ RESTful API cục bộ 30

Hình 4.2: Giao diện trang chủ thư viện ReadOwl Library (public/index.html) hiển thị danh mục sách, bộ lọc thể loại và tìm kiếm thời gian thực 31

Hình 4.3: Giao diện đăng nhập nhận mã thông báo JWT (public/login.html) và kiểm tra chuỗi Bearer Token trong DevTools LocalStorage 31

Hình 4.4: Giao diện trang quản lý phiếu mượn cá nhân của độc giả (public/my-loans.html) và chức năng đăng ký mượn / hủy phiếu mượn.....32

Hình 4.5: Giao diện Bảng điều khiển thống kê dành cho Quản trị viên (public/admin/dashboard.html) kết xuất dữ liệu từ GET /api/statistics/summary 32

Hình 4.6: Giao diện Quản trị kho sách (public/admin/books.html) và Điều phối phiếu mượn trả (public/admin/loans.html) của Quản trị viên.....33

Hình 4.7: Kết quả thực thi thành công 100% hai bộ kịch bản kiểm thử tự động (34/34 tests trong run-api-tests.php và 31/31 tests trong run-member2-tests.php)...33

Hình 4.8: Kết quả kiểm thử các điểm cuối RESTful API và kiểm tra cấu trúc phản hồi JSON trên công cụ Postman.....34

DANH MỤC BẢNG BIỂU

Bảng 1.1: Tổng quan bố cục và nội dung chính của các chương trong báo cáo.....3

Bảng 2.1: Ma trận ánh xạ phương thức HTTP với thao tác CRUD, tính an toàn và tính lũy đẳng trong hệ thống 6

Bảng 2.2: Phân loại các mã trạng thái HTTP tiêu chuẩn sử dụng trong hệ thống ReadOwl Library.....7

Bảng 2.3: So sánh đặc tính kỹ thuật và bảo mật giữa Session-Cookie và Bearer JWT phi trạng thái 10

Bảng 2.4: Ma trận so sánh tổng quan giữa RESTful API, GraphQL và gRPC 11

Bảng 3.1: Đặc tả các yêu cầu chức năng (FR) và yêu cầu phi chức năng (NFR) của hệ thống ReadOwl Library.....14

Bảng 3.2: Ngăn xếp công nghệ sử dụng trong hệ thống ReadOwl Library và lý giải lựa chọn 17

Bảng 3.3: Đặc tả cấu trúc các bảng dữ liệu, khóa ngoại và chỉ mục tối ưu hóa trong cơ sở dữ liệu library_rest.....19

Bảng 3.4: Bản đồ toàn bộ 25 điểm cuối RESTful API (API Endpoints Mapping) của hệ thống ReadOwl Library.....20

Bảng 3.5: Bảng ánh xạ giữa các nguyên lý lý thuyết RESTful Chương 2 và hiện thực mã nguồn trong dự án ReadOwl Library 24

Bảng 4.1: Thống kê bộ dữ liệu khởi tạo (Seed Dataset) và danh sách tài khoản kiểm thử trong database/seed.sql 25

Bảng 4.2: Ma trận 65 kịch bản kiểm thử tích hợp tự động trên hai bộ công cụ CLI Test Suites của dự án.....27

Bảng 4.3: Tổng hợp kết quả thực thi kiểm thử tích hợp và đo đạc thời gian phản hồi các điểm cuối RESTful API.....34

Bảng 4.4: Tổng hợp các thách thức kỹ thuật gặp phải trong quá trình thực nghiệm và giải pháp khắc phục trong mã nguồn 36

Bảng 5.1: Lộ trình các hướng phát triển mở rộng chức năng và nâng cấp kiến trúc cho hệ thống ReadOwl Library 39

DANH MỤC CÁC CHỮ VIẾT TẮT

Chữ viết tắt	Thuật ngữ tiếng Anh – Giải nghĩa tiếng Việt
ACID	Atomicity, Consistency, Isolation, Durability (Nguyên tử, Nhất quán, Cô lập, Bền vững)
API	Application Programming Interface (Giao diện lập trình ứng dụng)
BOLA	Broken Object Level Authorization (Lỗi kiểm soát quyền truy cập cấp đối tượng)
CLI	Command-Line Interface (Giao diện dòng lệnh)
CORS	Cross-Origin Resource Sharing (Chia sẻ tài nguyên giữa các nguồn khác nhau)
CRUD	Create, Read, Update, Delete (Tạo, Đọc, Cập nhật, Xóa)
CSDL	Cơ sở dữ liệu (Database)
CSS	Cascading Style Sheets (Ngôn ngữ định kiểu giao diện Web)
DOM	Document Object Model (Mô hình đối tượng tài liệu)
ERD	Entity-Relationship Diagram (Sơ đồ quan hệ thực thể)
FK	Foreign Key (Khóa ngoại)
FSM	Finite State Machine (Máy trạng thái hữu hạn)
HTML	HyperText Markup Language (Ngôn ngữ đánh dấu siêu văn bản)
HTTP	Hypertext Transfer Protocol (Giao thức truyền tải siêu văn bản)
HTTPS	Hypertext Transfer Protocol Secure (Giao thức HTTP bảo mật)
IDOR	Insecure Direct Object Reference (Tham chiếu đối tượng trực tiếp không an toàn)
ISBN	International Standard Book Number (Mã số sách tiêu chuẩn quốc tế)

JSON	JavaScript Object Notation (Định dạng trao đổi dữ liệu JSON)
JWT	JSON Web Token (Mã thông báo xác thực định dạng JSON)
OWASP	Open Worldwide Application Security Project (Dự án an ninh ứng dụng mở toàn cầu)
PDO	PHP Data Objects (Lớp trừu tượng truy cập CSDL trong PHP)
PK	Primary Key (Khóa chính)
RBAC	Role-Based Access Control (Kiểm soát truy cập dựa trên vai trò)
REST	Representational State Transfer (Chuyển giao trạng thái biểu diễn)
RFC	Request for Comments (Đặc tả tiêu chuẩn kỹ thuật Internet)
RMM	Richardson Maturity Model (Mô hình trưởng thành Richardson)
SPA	Single-Page Application (Ứng dụng Web đơn trang)
SQL	Structured Query Language (Ngôn ngữ truy vấn có cấu trúc)
TTL	Time-To-Live (Thời gian sống hiệu lực của mã thông báo)
URI	Uniform Resource Identifier (Định danh tài nguyên thống nhất)

CHƯƠNG 1. MỞ ĐẦU VÀ TỔNG QUAN ĐỀ TÀI

1.1 Lý do chọn đề tài

Trong các ứng dụng web truyền thống, giao diện HTML và logic xử lý nghiệp vụ thường được ghép chặt với nhau trên máy chủ (Server-Side Rendering). Mô hình này bộc lộ nhiều hạn chế khi hệ thống mở rộng, khó tái sử dụng logic nghiệp vụ cho các nền tảng khác nhau như ứng dụng di động hoặc đối tác tích hợp, đồng thời việc lưu trạng thái phiên làm việc (Session) trên bộ nhớ máy chủ gây khó khăn lớn cho việc mở rộng quy mô hệ thống.

Kiến trúc Dịch vụ Web RESTful (Representational State Transfer) do Roy Fielding đề xuất đã trở thành tiêu chuẩn cốt lõi để giải quyết triệt để vấn đề trên. Bằng cách trừu tượng hóa các thực thể thành tài nguyên (Resource), định danh qua URI và thao tác thông qua các phương thức HTTP chuẩn (GET, POST, PUT, DELETE) với định dạng JSON gọn nhẹ, kiến trúc REST tách biệt hoàn toàn giữa máy khách (Frontend) và máy chủ (Backend). Nhờ nguyên lý phi trạng thái (Stateless), máy chủ không cần lưu session, giúp tăng tối đa khả năng mở rộng và độ ổn định.

[VỊ TRÍ CHÈN HÌNH 1.1: SỰ CHUYỂN DỊCH TỪ KIẾN TRÚC WEB NGUYÊN KHỐI SANG KIẾN TRÚC RESTFUL CLIENT–SERVER TÁCH BIỆT]

Hướng dẫn: Vẽ sơ đồ đối chiếu giữa: (1) Kiến trúc Web nguyên khối truyền thống (Browser gửi Request, Web Server xử lý PHP ghép Session và HTML trả về toàn bộ trang) và (2) Kiến trúc RESTful hiện đại (Frontend Client gửi HTTP Request kèm JWT, Backend REST API xử lý trả về dữ liệu thô JSON)

Hình 1.1: Sự chuyển dịch từ kiến trúc Web nguyên khối sang kiến trúc RESTful Client–Server tách biệt

(Nguồn: Nhóm tác giả tự tổng hợp)

Nhằm nắm vững nguyên lý hoạt động của kiến trúc REST và áp dụng vào bài toán thực tế, nhóm lựa chọn thực hiện đề tài nghiên cứu kiến trúc RESTful Web Services và xây dựng ứng dụng thực nghiệm: Hệ thống quản lý thư viện mượn – trả sách ReadOwl Library. Hệ thống thư viện là mô hình thực nghiệm tiêu biểu vì bao gồm cả tài nguyên công khai (tra cứu sách, thể loại, phân trang, tìm kiếm) lẫn tài

nguyên nhạy cảm đòi hỏi phân quyền chặt chẽ (quản lý người dùng, tạo phiếu mượn, cập nhật trạng thái mượn trả, kiểm soát số lượng sách tồn kho và báo cáo thống kê).

1.2 Mục tiêu thực hiện đề tài

1.2.1 Mục tiêu nghiên cứu lý thuyết và phát triển thực nghiệm

Về lý thuyết, đề tài hướng đến việc làm rõ các nguyên lý cốt lõi của kiến trúc REST: 4 ràng buộc kiến trúc áp dụng thực tế, mô hình trưởng thành Richardson (RMM), ngữ nghĩa các phương thức HTTP, quy chuẩn mã trạng thái phản hồi, cấu trúc mã thông báo JSON Web Token (JWT) và các nguyên tắc thiết kế API an toàn theo chuẩn OWASP.

Về thực nghiệm, nhóm đặt mục tiêu xây dựng hoàn chỉnh hệ thống ReadOwl Library bằng PHP 8.2 thuần kết hợp CSDL MySQL: cung cấp 25 điểm cuối RESTful API chuẩn mực, xử lý giao dịch mượn trả an toàn bằng PDO Transaction và khóa dòng (SELECT ... FOR UPDATE), xác thực Bearer JWT HS256, xây dựng giao diện người dùng bất đồng bộ bằng Vanilla JS Fetch API, tài liệu hóa toàn bộ bằng OpenAPI 3.0.3 và kiểm chứng độ tin cậy bằng 65 kịch bản kiểm thử tự động.

1.2.2 Phạm vi thực hiện và đối tượng sử dụng hệ thống

Hệ thống ReadOwl Library tập trung vào quy trình quản lý danh mục sách và nghiệp vụ mượn – trả sách vật lý, phục vụ ba nhóm đối tượng người dùng: Khách vãng lai (Guest - tra cứu, tìm kiếm, xem chi tiết sách), Độc giả (Reader - đăng ký, đăng nhập, quản lý hồ sơ cá nhân, tạo phiếu mượn sách trong hạn mức 3 cuốn, theo dõi và hủy phiếu mượn của mình) và Quản trị viên (Admin - quản trị danh mục sách, thể loại, tài khoản độc giả, duyệt trả sách, đánh dấu quá hạn và xem thống kê báo cáo).

1.2.3 Bố cục của báo cáo

Báo cáo được tổ chức thành 5 chương bám sát đúng khuôn mẫu quy định của Khoa Công nghệ thông tin như trình bày trong Bảng 1.1.

Bảng 1.1: Tổng quan bố cục và nội dung chính của các chương trong báo cáo

Chương / Phần	Tên chương / phần	Nội dung trọng tâm thực hiện
Chương 1	Mở đầu và Tổng quan đề tài	Lý do chọn đề tài, mục tiêu nghiên cứu và phát triển, phạm vi và bố cục báo cáo.
Chương 2	Cơ sở lý thuyết	Khảo sát nguyên lý REST, mô hình Richardson, chuẩn HTTP RFC 9110, xác thực JWT HS256 và bảo mật OWASP.
Chương 3	Mô hình đề xuất	Phân tích yêu cầu, kiến trúc 4 tầng, thiết kế CSDL 3NF, hợp đồng 25 endpoints REST API và các đoạn mã nguồn cốt lõi.
Chương 4	Thực nghiệm	Bộ dữ liệu mẫu, 65 ca kiểm thử tự động CLI, ảnh chụp màn hình demo các chức năng, kết quả đo đạc và cách giải quyết thách thức.
Chương 5	Kết luận	Tổng kết kết quả đạt được và định hướng nâng cấp hệ thống trong tương lai.
Phụ lục	Phân công nhiệm vụ & Thông tin Demo	Bảng phân chia công việc 50/50, thông tin tài khoản kiểm thử và 5 lệnh vận hành nhanh.

(Nguồn: Nhóm tác giả tự tổng hợp, 2026)

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Lịch sử hình thành và nguyên lý cốt lõi của kiến trúc REST

2.1.1 Bối cảnh ra đời của dịch vụ Web và luận án của Roy Fielding

Phong cách kiến trúc REST (Representational State Transfer) được Roy Thomas Fielding giới thiệu lần đầu tiên vào năm 2000 trong luận án tiến sĩ tại Đại học California, Irvine. Khác với các mô hình gọi thủ tục từ xa (RPC) vốn coi mạng máy tính như một phần mở rộng của bộ nhớ cục bộ và bị phụ thuộc chặt vào công nghệ máy chủ, REST khai thác chính các đặc tính sẵn có của giao thức HTTP trên World Wide Web để định nghĩa một tập các ràng buộc kiến trúc giúp các hệ thống phân tán giao tiếp mềm dẻo và mở rộng dễ dàng.

2.1.2 Bốn ràng buộc kiến trúc cốt lõi áp dụng trong hệ thống

Mặc dù lý thuyết REST quy định sáu ràng buộc, trong việc xây dựng các ứng dụng web hiện đại, bốn ràng buộc sau đây mang ý nghĩa quyết định và được áp dụng triệt để trong dự án ReadOwl Library:

1. Phân tách Máy khách – Máy chủ (Client–Server Separation): Tách biệt hoàn toàn giao diện người dùng khỏi tầng lưu trữ và xử lý dữ liệu. Giao diện khách (HTML/JS) có thể thay đổi hoặc nâng cấp độc lập mà không ảnh hưởng tới API Backend, và ngược lại Backend có thể phục vụ đồng thời nhiều nền tảng khách khác nhau thông qua một giao diện dữ liệu duy nhất.

2. Phi trạng thái (Statelessness): Mỗi yêu cầu từ máy khách gửi lên máy chủ phải chứa đầy đủ mọi thông tin cần thiết để máy chủ hiểu và xử lý (bao gồm thông tin xác thực danh tính qua Bearer JWT). Máy chủ tuyệt đối không lưu trạng thái phiên làm việc (Session) của người dùng trên bộ nhớ giữa các yêu cầu. Điều này giúp giải phóng tài nguyên máy chủ và cho phép dễ dàng cân bằng tải ngang qua nhiều máy chủ API.

3. Khả năng lưu đệm (Cacheability): Các phản hồi từ API phải được dán nhãn rõ ràng có thể lưu đệm được hay không (thông qua tiêu đề HTTP Cache-Control,

ETag). Các yêu cầu đọc dữ liệu ít biến động như danh mục thể loại hoặc chi tiết sách có thể được lưu đệm tại máy khách hoặc trung gian, giúp giảm đáng kể độ trễ phản hồi và giảm tải truy vấn cơ sở dữ liệu.

4. Giao diện đồng nhất (Uniform Interface): Đây là ràng buộc đặc trưng nhất của REST, yêu cầu mọi tài nguyên phải được định danh qua URI chuẩn, thao tác thông qua các phương thức HTTP chuẩn (GET, POST, PUT, DELETE), và định dạng dữ liệu trả về phải nhất quán (chuẩn hóa bằng định dạng JSON).

2.1.3 Tài nguyên, biểu diễn tài nguyên và mô hình trưởng thành Richardson

Trong REST, mọi thực thể thông tin quan trọng đều được trừu tượng hóa thành 'Tài nguyên' (Resource) và được định danh bằng URI (ví dụ: /api/books/1 đại diện cho đầu sách có mã số 1). Khi tương tác, máy khách không truy cập trực tiếp vào bản ghi cơ sở dữ liệu mà nhận được một 'Biểu diễn' (Representation) của tài nguyên đó dưới định dạng tài liệu JSON.

Để đánh giá mức độ tuân thủ chuẩn REST của một API, Leonard Richardson đã đề xuất Mô hình trưởng thành Richardson (Richardson Maturity Model – RMM) gồm 4 cấp độ như mô tả tại Hình 2.1:

[VỊ TRÍ CHÈN HÌNH 2.1: MÔ HÌNH TRƯỞNG THÀNH RICHARDSON (RICHARDSON MATURITY MODEL) VÀ VỊ TRÍ HIỆN THỰC CỦA DỰ ÁN READOWL LIBRARY]

Hướng dẫn: Vẽ sơ đồ 4 cấp độ kim tự tháp của Mô hình trưởng thành Richardson: Level 0 (The Swamp of POX - dùng 1 endpoint POST duy nhất kiểu RPC); Level 1 (Resources - định danh tài nguyên bằng URI riêng biệt); Level 2 (HTTP Verbs & Status Codes - sử dụng đúng GET/POST/PUT/DELETE và mã trạng thái chuẩn); Level 3 (HATEOAS - nhúng siêu liên kết điều hướng trong phản hồi). Đánh dấu rõ ReadOwl Library đạt chuẩn Level 2 hoàn chỉnh

Hình 2.1: Mô hình trưởng thành Richardson (Richardson Maturity Model) và vị trí hiện thực của dự án ReadOwl Library

(Nguồn: Nhóm tác giả tổng hợp từ Martin Fowler, 2010)

Dự án ReadOwl Library đạt chuẩn Cấp độ 2 (Level 2) hoàn chỉnh: định danh rõ ràng từng tài nguyên bằng danh từ số nhiều (/api/books, /api/loans), sử dụng đúng ngữ nghĩa động từ HTTP cho từng thao tác, và phản hồi chính xác bằng các mã trạng

thái HTTP chuẩn (200, 201, 400, 401, 403, 404, 409, 422). Cấp độ 3 (HATEOAS) được nhóm xác định là hướng mở rộng trong tương lai.

2.2 Đặc tả kỹ thuật giao thức HTTP và thực tiễn thiết kế RESTful API

2.2.1 Ngữ nghĩa phương thức HTTP, ánh xạ CRUD, tính an toàn và tính lũy đẳng

Kiến trúc REST tận dụng trực tiếp ngữ nghĩa của giao thức HTTP theo chuẩn RFC 9110 thay vì định nghĩa các tên hàm tùy biến. Hai thuộc tính kỹ thuật quan trọng nhất cần tuân thủ là:

- Tính an toàn (Safe): Phương thức chỉ thực hiện thao tác đọc, không làm thay đổi trạng thái của tài nguyên trên máy chủ. Máy khách có thể gọi lặp lại nhiều lần mà không gây ra tác dụng phụ (Side Effect).

- Tính lũy đẳng (Idempotent): Thực thi yêu cầu một lần hay nhiều lần liên tiếp với cùng dữ liệu đầu vào đều mang lại kết quả trạng thái tài nguyên trên máy chủ như nhau.

Bảng 2.1 tổng hợp ma trận ánh xạ giữa các phương thức HTTP với các thao tác CRUD trong hệ thống ReadOwl Library.

Bảng 2.1: Ma trận ánh xạ phương thức HTTP với thao tác CRUD, tính an toàn và tính lũy đẳng trong hệ thống

Phương thức HTTP	Thao tác CRUD	Tính an toàn (Safe)	Tính lũy đẳng (Idempotent)	Ứng dụng cụ thể trong ReadOwl Library
GET	Read (Đọc)	Có (Safe)	Có (Idempotent)	Tra cứu danh sách sách (GET /api/books), xem chi tiết (GET /api/books/{id}), xem hồ sơ.
POST	Create (Tạo mới)	Không	Không	Đăng ký tài khoản (POST /api/auth/register), đăng nhập nhận JWT, tạo phiếu mượn mới.
PUT	Update (Cập nhật)	Không	Có (Idempotent)	Cập nhật toàn bộ thông tin sách (PUT /api/books/{id}), cập nhật trạng thái phiếu mượn.

Phương thức HTTP	Thao tác CRUD	Tính an toàn (Safe)	Tính lũy đẳng (Idempotent)	Ứng dụng cụ thể trong ReadOwl Library
DELETE	Delete (Xóa)	Không	Có (Idempotent)	Xóa thể loại sách rỗng (DELETE /api/categories/{id}), xóa tài khoản chưa có giao dịch.

(Nguồn: Nhóm tác giả tự tổng hợp, 2026)

2.2.2 Cấu trúc thông điệp HTTP và hệ thống mã trạng thái chuẩn

Mỗi yêu cầu và phản hồi HTTP bao gồm ba phần: Dòng khởi đầu (Start Line), Khối tiêu đề (Headers) và Thân thông điệp (Body). Trong ReadOwl Library, toàn bộ API giao tiếp qua tiêu đề chuẩn: máy khách gửi Accept: application/json và Content-Type: application/json; máy chủ phản hồi application/json; charset=utf-8 kèm các mã trạng thái phản ánh chính xác kết quả nghiệp vụ như tổng hợp tại Bảng 2.2.

Bảng 2.2: Phân loại các mã trạng thái HTTP tiêu chuẩn sử dụng trong hệ thống ReadOwl Library

Mã HTTP	Tên chuẩn tiếng Anh	Ý nghĩa nghiệp vụ	Tình huống áp dụng thực tế trong ReadOwl Library
200 OK	Success	Thao tác thành công	Lấy danh sách sách, xem chi tiết, cập nhật hồ sơ cá nhân thành công.
201 Created	Created	Tạo tài nguyên mới thành công	Đăng ký tài khoản thành công, tạo phiếu mượn sách thành công, thêm đầu sách mới.
400 Bad Request	Client Error	Cú pháp yêu cầu không hợp lệ	Dữ liệu JSON gửi lên bị sai cú pháp hoặc thiếu thông tin bắt buộc cơ bản.
401 Unauthorized	Authentication Required	Chưa xác thực hoặc token không hợp lệ	Thiếu tiêu đề Authorization, chuỗi JWT sai chữ ký hoặc token đã hết hạn 900 giây.
403 Forbidden	Access Denied	Không có quyền thực hiện	Tài khoản Độc giả cố tình gọi API xóa sách của Admin, hoặc tài khoản đang bị khóa.
404 Not Found	Not Found	Không tìm thấy tài nguyên	Tra cứu mã đầu sách hoặc phiếu mượn không tồn tại trong cơ sở dữ liệu.
409 Conflict	Conflict	Xung đột trạng thái tài nguyên	Đăng ký trùng email, sách đã hết bản sao khả dụng (available_quantity = 0), hoặc độc giả đã mượn tối đa 3 cuốn.

Mã HTTP	Tên chuẩn tiếng Anh	Ý nghĩa nghiệp vụ	Tình huống áp dụng thực tế trong ReadOwl Library
422 Unproc. Entity	Validation Error	Lỗi kiểm tra tính hợp lệ dữ liệu	Mật khẩu ngắn hơn 6 ký tự, ngày hẹn trả nhỏ hơn ngày mượn, định dạng email sai.
500 Internal Error	Server Error	Lỗi nội bộ máy chủ	Mất kết nối CSDL MySQL hoặc ngoại lệ hệ thống không lường trước.

(Nguồn: Nhóm tác giả tự tổng hợp, 2026)

2.2.3 Thực tiễn thiết kế URI, phân trang và chuẩn hóa hợp đồng OpenAPI 3.0.3

Hệ thống ReadOwl Library áp dụng các quy chuẩn thiết kế API hiện đại:

1. Thiết kế URI: Sử dụng danh từ số nhiều viết thường (/api/books, /api/loans) đại diện cho tập tài nguyên; sử dụng định danh ID trên đường dẫn cho tài nguyên cụ thể (/api/books/{id}); biểu diễn mối quan hệ phụ thuộc qua phân cấp (/api/categories/{id}/books); tuyệt đối không dùng động từ trên URI.

2. Phân trang và tìm kiếm: Đối với tập dữ liệu lớn, sử dụng tham số truy vấn (Query Parameters) như ?page=1&limit=10, ?q=php, ?category_id=2. Dữ liệu phản hồi luôn kèm đối tượng siêu dữ liệu meta.pagination chứa tổng số bản ghi (total_items), tổng số trang (total_pages) và trang hiện tại.

3. Chuẩn hóa phong bì JSON (Uniform JSON Envelope): Mọi điểm cuối đều phản hồi theo một cấu trúc thống nhất: { success: true/false, message: '...', data: ..., meta: ..., errors: ... }, giúp máy khách xử lý kết quả đồng bộ và dễ dàng bắt lỗi.

4. Đặc tả hợp đồng OpenAPI 3.0.3: Toàn bộ 25 điểm cuối, tham số đầu vào, lược đồ JSON và mã phản hồi được chuẩn hóa độc lập trong tệp docs/openapi.yaml, cho phép kiểm thử trực quan trên Swagger UI / Postman và làm căn cứ đối chiếu kiểm thử tự động.

2.3 Cơ chế bảo mật và xác thực trong dịch vụ Web RESTful

2.3.1 Cấu trúc JSON Web Token (JWT) và cơ chế xác thực chữ ký số HMAC-SHA256

Nhằm đảm bảo nguyên lý phi trạng thái (Stateless), hệ thống ReadOwl Library sử dụng tiêu chuẩn JSON Web Token (JWT - RFC 7519) kết hợp thư viện chuẩn công nghiệp firebase/php-jwt. Chuỗi mã thông báo JWT bao gồm ba phần được phân tách bằng dấu chấm (.):

- Header: Chứa kiểu token (typ: 'JWT') và thuật toán ký mã hóa (alg: 'HS256').

- Payload: Chứa các thông tin ngữ cảnh (Claims) bao gồm định danh người dùng (sub), email, vai trò (role: 'admin' hoặc 'reader'), thời điểm phát hành (iat) và thời điểm hết hạn (exp = 900 giây).

- Signature: Chữ ký số được tính toán bằng thuật toán HMAC-SHA256 trên chuỗi kết hợp (Header + Payload) với một khóa bí mật (Secret Key) lưu an toàn trên máy chủ. Máy khách không thể giả mạo thông tin bên trong Payload vì bất kỳ sự thay đổi nào cũng làm sai lệch chữ ký.

[VỊ TRÍ CHÈN HÌNH 2.2: CẤU TRÚC BA THÀNH PHẦN CỦA JSON WEB TOKEN VÀ QUY TRÌNH XÁC THỰC BEARER TOKEN PHI TRẠNG THÁI]

Hướng dẫn: Vẽ sơ đồ cấu trúc 3 phần của JWT: Header (Base64Url) . Payload (Base64Url) . Signature (HMAC-SHA256) và quy trình gửi token qua HTTP Header 'Authorization: Bearer <token>' từ Browser đến REST API

Hình 2.2: Cấu trúc ba thành phần của JSON Web Token và quy trình xác thực Bearer Token phi trạng thái

(Nguồn: Nhóm tác giả tổng hợp từ RFC 7519)

2.3.2 So sánh cơ chế xác thực Session-Cookie và Bearer Token phi trạng thái

Bảng 2.3 phân tích lý do hệ thống lựa chọn Bearer JWT thay vì cơ chế Session-Cookie truyền thống cho dịch vụ RESTful API.

Bảng 2.3: So sánh đặc tính kỹ thuật và bảo mật giữa Session-Cookie và Bearer JWT phi trạng thái

Tiêu chí so sánh	Cơ chế Session-Cookie truyền thống	Cơ chế Bearer JWT (Áp dụng trong ReadOwl)
Lưu trữ trạng thái	Lưu trên bộ nhớ RAM hoặc tệp tin của máy chủ API.	Hoàn toàn phi trạng thái (Stateless); trạng thái nằm trọn vẹn trong token tại Client.
Khả năng mở rộng	Khó khăn khi mở rộng ngang; cần Redis tập trung hoặc cấu hình Sticky Session.	Cực kỳ dễ dàng mở rộng; mọi máy chủ API đều tự xác minh token độc lập qua Secret Key.
Bảo mật liên trang (CSRF)	Dễ bị tấn công CSRF do trình duyệt tự động gửi kèm Cookie theo mặc định.	Miễn nhiễm với CSRF vì token lưu trong Web Storage và chỉ gửi qua tiêu đề Authorization.
Hỗ trợ đa nền tảng	Thích hợp cho trình duyệt Web; phức tạp khi kết nối với Mobile App hoặc Third-party.	Chuẩn hóa hoàn hảo cho mọi nền tảng (Web SPA, Mobile iOS/Android, Backend Services).

(Nguồn: Nhóm tác giả tự tổng hợp, 2026)

2.3.3 Các mối đe dọa bảo mật API phổ biến theo OWASP và chiến lược phòng vệ

Dựa trên tài liệu OWASP API Security Top 10, nhóm đã chủ động thiết kế các cơ chế phòng vệ tương ứng trong mã nguồn dự án:

- Lỗi kiểm soát truy cập đối tượng (BOLA/IDOR): Độc giả chỉ được phép xem và hủy phiếu mượn của chính mình. Tại các API /api/loans/{id}, mô-đun loans kiểm tra user_id trong JWT khớp với user_id của phiếu mượn (trừ khi người gọi là Admin).

- Lỗi xác thực (Broken Authentication): Mật khẩu người dùng được băm một chiều bằng thuật toán Bcrypt mạnh mẽ (password_hash) với cost = 12. Thời hạn JWT được đặt ngắn (900 giây / 15 phút) để hạn chế rủi ro lộ token.

- Tấn công tiêm mã SQL (SQL Injection): 100% các câu truy vấn cơ sở dữ liệu đều sử dụng kỹ thuật tham số hóa (Prepared Statements) của PDO với cờ PDO::ATTR_EMULATE_PREPARES = false, đảm bảo câu lệnh SQL được phân tích cú pháp trước khi dữ liệu người dùng được chèn vào.

- Tấn công chiếm dụng tài nguyên (Denial of Service): Các API danh sách luôn bắt buộc giới hạn số lượng bản ghi trả về (mặc định limit=10, tối đa limit=50) để tránh làm sập bộ nhớ máy chủ khi truy vấn.

2.4 Phân tích ưu nhược điểm, thách thức kỹ thuật và so sánh các kiến trúc dịch vụ Web

2.4.1 Điểm mạnh, điểm yếu và giải pháp giảm thiểu trong dự án

Kiến trúc REST mang lại ưu điểm vượt trội về tính đơn giản, khả năng mở rộng và tính tương thích cao với hạ tầng Web sẵn có. Tuy nhiên, kiến trúc này cũng tồn tại hai nhược điểm cố hữu: hiện tượng lấy dư hoặc thiếu dữ liệu (Over-fetching / Under-fetching) và sự phức tạp trong việc thu hồi ngay lập tức một token JWT phi trạng thái khi tài khoản bị khóa. Trong dự án ReadOwl Library, nhóm khắc phục bằng cách thiết kế các endpoint có trường dữ liệu tinh gọn phù hợp giao diện, đồng thời cài đặt hàm kiểm tra trạng thái is_active của người dùng trong cơ sở dữ liệu ngay khi giải mã JWT để chặn tức thì các tài khoản bị khóa.

2.4.2 So sánh tổng quan giữa RESTful API, GraphQL và gRPC

Bảng 2.4 so sánh REST với hai phong cách giao tiếp dịch vụ web phổ biến khác là GraphQL và gRPC, làm rõ lý do RESTful API là lựa chọn phù hợp nhất cho hệ thống quản lý thư viện học thuật ReadOwl Library.

Bảng 2.4: Ma trận so sánh tổng quan giữa RESTful API, GraphQL và gRPC

Tiêu chuẩn kỹ thuật	RESTful API (Lựa chọn của dự án)	GraphQL	gRPC
Mô hình kiến trúc	Định hướng tài nguyên (Resource-oriented)	Định hướng đồ thị truy vấn (Graph-oriented)	Định hướng gọi thủ tục từ xa (RPC-oriented)
Giao thức & Định dạng	HTTP/1.1 hoặc HTTP/2; định dạng JSON chuẩn	HTTP POST; định dạng JSON	Bắt buộc HTTP/2; nhị phân Protocol Buffers
Cơ chế lưu đệm (Cache)	Tận dụng đệm HTTP gốc (Browser, Proxy, CDN)	Phức tạp vì chỉ dùng POST một endpoint duy nhất	Không hỗ trợ đệm HTTP chuẩn

Tiêu chuẩn kỹ thuật	RESTful API (Lựa chọn của dự án)	GraphQL	gRPC
Độ phức tạp cài đặt	Thấp; dễ xây dựng bằng PHP thuần và dễ gỡ lỗi	Cao; đòi hỏi Schema, Resolver và thư viện phức tạp	Cao; đòi hỏi biên dịch tệp .proto và môi trường đặc thù
Trường hợp tối ưu	Hệ thống Web CRUD công cộng, thư viện, thương mại điện tử	Ứng dụng di động nhiều màn hình dữ liệu phức tạp	Giao tiếp nội bộ giữa các vi dịch vụ (Microservices)

(Nguồn: Nhóm tác giả tự tổng hợp, 2026)

CHƯƠNG 3. MÔ HÌNH ĐỀ XUẤT

3.1 Tổng quan hệ thống ReadOwl Library và phân tích yêu cầu

3.1.1 Mục đích hệ thống, đối tượng người dùng mục tiêu và các tính năng cốt lõi

Để hiện thực hóa các nguyên lý kiến trúc RESTful đã nghiên cứu, nhóm xây dựng hệ thống quản lý thư viện trực tuyến ReadOwl Library. Hệ thống số hóa quy trình quản lý danh mục ấn phẩm, kiểm soát tồn kho bản sao vật lý và điều phối vòng đời mượn – trả sách. Hệ thống áp dụng mô hình tách biệt hoàn toàn giữa giao diện người dùng (Frontend) và dịch vụ cung cấp dữ liệu (Backend REST API) qua giao thức HTTP và định dạng dữ liệu chuẩn JSON.

Hệ thống phục vụ ba nhóm tác nhân với phân quyền chặt chẽ: Khách vãng lai (Guest - tra cứu danh mục, tìm kiếm sách, lọc theo thể loại và kiểm tra số lượng bản sao còn sẵn sàng); Độc giả thành viên (Reader - đăng ký, đăng nhập nhận Bearer JWT, quản lý hồ sơ cá nhân, gửi yêu cầu mượn sách trong hạn mức 3 cuốn, theo dõi lịch sử và chủ động hủy phiếu mượn); và Quản trị viên (Admin - quản trị danh mục thể loại, kho sách, tài khoản độc giả, duyệt mượn, nhận trả sách, đánh dấu quá hạn và theo dõi thống kê thư viện thời gian thực).

3.1.2 Đặc tả yêu cầu chức năng và yêu cầu phi chức năng

Hệ thống được đặc tả thành hai nhóm yêu cầu kỹ thuật: Yêu cầu chức năng (FR) định nghĩa hành vi nghiệp vụ và Yêu cầu phi chức năng (NFR) quy định ràng buộc về kiến trúc, an ninh và toàn vẹn dữ liệu như tổng hợp tại Bảng 3.1.

Bảng 3.1: Đặc tả các yêu cầu chức năng (FR) và yêu cầu phi chức năng (NFR) của hệ thống ReadOwl Library

Mã YC	Nhóm yêu cầu	Tên yêu cầu	Mô tả chi tiết và tiêu chí nghiệm thu kỹ thuật
FR-01	Xác thực & Hồ sơ	Đăng ký, đăng nhập và quản lý hồ sơ	Đăng ký tài khoản độc giả (POST /api/auth/register), đăng nhập nhận JWT (POST /api/auth/login), xem và cập nhật thông tin cá nhân, đổi mật khẩu (GET, PUT /api/auth/me).
FR-02	Thể loại sách	Tra cứu và quản trị thể loại	Liệt kê và xem chi tiết thể loại công khai (GET /api/categories). Quản trị viên thêm mới, sửa và xóa thể loại rỗng chưa có sách (POST, PUT, DELETE /api/categories/{id}).
FR-03	Kho đầu sách	Tìm kiếm, lọc, phân trang và quản trị	Tìm kiếm sách theo từ khóa, lọc theo thể loại, lọc tồn kho khả dụng và phân trang (GET /api/books). Quản trị viên thêm sách, cập nhật số lượng và xóa sách chưa có giao dịch.
FR-04	Mượn – Trả sách	Tạo phiếu mượn và điều phối trạng thái	Độc giả tạo yêu cầu mượn sách khi còn tồn kho và chưa vượt hạn mức 3 cuốn (POST /api/loans). Quản trị viên duyệt trả, đánh dấu quá hạn hoặc hủy phiếu theo FSM và tự động đồng bộ tồn kho.
FR-05	Người dùng	Quản lý tài khoản độc giả	Admin tra cứu danh sách người dùng, xem chi tiết lịch sử mượn, cập nhật vai trò, khóa/mở khóa tài khoản (is_active) và xóa tài khoản chưa phát sinh giao dịch.
FR-06	Báo cáo thống kê	Tổng hợp số liệu vận hành thư viện	Tổng hợp số lượng đầu sách, bản sao, độc giả, phiếu mượn theo trạng thái, Top sách mượn nhiều nhất và danh sách phiếu mượn quá hạn (GET /api/statistics/summary).
NFR-01	Kiến trúc REST	Phi trạng thái (Stateless) & Phong bì JSON	Không dùng PHP $_SESSION. Mọi phản hồi HTTP trả đúng mã trạng thái chuẩn (200, 201, 400, 401, 403, 404, 409, 422) và bọc trong cấu trúc JSON đồng nhất.
NFR-02	Bảo mật	JWT HS256 & Băm mật khẩu Bcrypt	Mật khẩu băm bằng Bcrypt. Xác thực yêu cầu qua tiêu đề Authorization: Bearer <JWT> có thời hạn 900s. Toàn bộ câu lệnh SQL sử dụng PDO Prepared Statements với EMULATE_PREPARES = false.

Mã YC	Nhóm yêu cầu	Tên yêu cầu	Mô tả chi tiết và tiêu chí nghiệm thu kỹ thuật
NFR-03	Toàn vẹn CSDL	Giao dịch ACID & Khóa dòng chống xung đột	Mọi thao tác mượn/trả sách thay đổi đồng thời bảng loans và available_quantity của bảng books đều thực thi trong PDO Transaction với khóa dòng SELECT ... FOR UPDATE.
NFR-04	Kiểm thử & Hợp đồng	Đặc tả OpenAPI 3.0.3 & Kiểm thử tự động	100% các điểm cuối được mô tả bằng OpenAPI 3.0.3 và kiểm chứng bằng 65 kịch bản kiểm thử tích hợp tự động trên CLI cùng bộ sưu tập Postman.

(Nguồn: Nhóm tác giả tự tổng hợp)

3.1.3 Mô hình hóa ca sử dụng (Use Case Modeling)

Sự tương tác giữa các tác nhân (Guest, Reader, Admin) và các phân hệ chức năng của hệ thống được mô hình hóa qua sơ đồ Use Case tại Hình 3.1. Độc giả kế thừa toàn bộ quyền tra cứu công khai của Khách vãng lai và mở rộng thêm các chức năng cần xác thực JWT. Quản trị viên nắm giữ toàn quyền quản trị danh mục, tài khoản và điều phối mượn trả.

[VỊ TRÍ CHÈN HÌNH 3.1: SƠ ĐỒ USE CASE TỔNG THỂ CỦA HỆ THỐNG QUẢN LÝ THƯ VIỆN READOWL LIBRARY]

Hướng dẫn: Vẽ sơ đồ Use Case UML gồm 3 Actor: Khách vãng lai (Guest), Độc giả (Reader - kế thừa Guest), và Quản trị viên (Admin). Thể hiện các Use Case: Tra cứu & lọc sách, Xem chi tiết sách & thể loại, Đăng ký & Đăng nhập nhận JWT, Quản lý hồ sơ cá nhân, Tạo phiếu mượn sách, Theo dõi & hủy phiếu mượn, Quản trị sách (CRUD), Quản trị thể loại (CRUD), Quản trị người dùng, Điều phối trạng thái mượn trả và Xem thống kê

Hình 3.1: Sơ đồ Use Case tổng thể của hệ thống quản lý thư viện ReadOwl Library

(Nguồn: Nhóm tác giả tự thiết kế)

3.2 Kiến trúc hệ thống và lựa chọn ngăn xếp công nghệ

3.2.1 Kiến trúc tổng thể bốn tầng tách biệt và nguyên lý phối hợp các thành phần

ReadOwl Library được xây dựng theo kiến trúc 4 tầng phân tách độc lập (Hình 3.2): (1) Tầng Client (HTML5/CSS3/Vanilla JS Fetch API, lưu JWT trong LocalStorage); (2) Tầng Điều phối (router.php/.htaccess chuyển tiếp vào Front

Controller api/index.php kết nối các middleware dùng chung request, auth, response, db); (3) Tầng Nghiệp vụ (api/modules gồm 6 phân hệ: auth, users, categories, books, loans, statistics); (4) Tầng Lưu trữ (CSDL MySQL InnoDB library_rest).

[VỊ TRÍ CHÈN HÌNH 3.2: SƠ ĐỒ KIẾN TRÚC TỔNG THỂ 4 TẦNG CỦA HỆ THỐNG READOWL LIBRARY VÀ LUỒNG XỬ LÝ YÊU CẦU RESTFUL API]

Hướng dẫn: Vẽ sơ đồ khối kiến trúc 4 tầng của dự án: (1) Tầng Client (Trình duyệt HTML/CSS/JS, public/assets/js/api.js, Postman/CLI Tests); (2) Tầng Điều phối trung tâm (router.php / .htaccess chuyển tiếp vào Front Controller api/index.php, kết nối 4 tiện ích trong api/common: request, auth, response, db); (3) Tầng Mô-đun nghiệp vụ (api/modules/: auth, users, categories, books, loans, statistics); (4) Tầng CSDL MySQL/MariaDB (library_rest với 4 bảng: users, categories, books, loans)

Hình 3.2: Sơ đồ kiến trúc tổng thể 4 tầng của hệ thống ReadOwl Library và luồng xử lý yêu cầu RESTful API

(Nguồn: Nhóm tác giả tự thiết kế)

Tại Tầng Điều phối, Front Controller (api/index.php) bóc tách đường dẫn URI thành các phân đoạn để chuyển tiếp yêu cầu đến kịch bản xử lý phù hợp mà không cần phụ thuộc vào framework nặng nề:

```
// api/index.php: Bóc tách đường dẫn URI và điều phối phân hệ RESTful
$uriPath = parse_url($_SERVER['REQUEST_URI'] ?? '/', PHP_URL_PATH) ?: '/';
$routePath = '';
if (preg_match('#/api(?:/(.*))?$#', $uriPath, $matches)) {
    $routePath = isset($matches[1]) ? trim($matches[1], '/') : '';
}
$segments = $routePath === '' ? [] : explode('/', $routePath);
$resource = $segments[0] ?? 'health';
$subSegment = $segments[1] ?? null;

// Chuyển tiếp yêu cầu đến module nghiệp vụ tương ứng
$moduleFile = __DIR__ . "/modules/{$resource}/routes.php";
if (file_exists($moduleFile)) {
    require $moduleFile;
} else {
    sendError(404, 'NOT_FOUND', 'Không tìm thấy phân hệ tài nguyên yêu
cầu.');
}
```

3.2.2 Lựa chọn ngăn xếp công nghệ và lý giải kiến trúc

Thay vì sử dụng các framework PHP cỡ lớn như Laravel hay Symfony vốn che khuất cơ chế hoạt động của giao thức HTTP, nhóm quyết định xây dựng hệ thống bằng Native PHP 8.2 kết hợp CSDL MySQL và thư viện firebase/php-jwt. Bảng 3.2 tổng hợp chi tiết các thành phần trong ngăn xếp công nghệ và lý do lựa chọn.

Bảng 3.2: Ngăn xếp công nghệ sử dụng trong hệ thống ReadOwl Library và lý giải lựa chọn

Tầng kiến trúc	Công nghệ / Thư viện	Phiên bản / Chuẩn	Vai trò và lý do lựa chọn trong kiến trúc RESTful
Ngôn ngữ Backend	Native PHP	>= 8.1 (Thực tế: PHP 8.2)	Minh họa trực quan cách thức tiếp nhận HTTP request, đọc php://input, cấu hình CORS và trả về JSON thuần mà không bị che khuất bởi framework.
Xác thực phi trạng thái	firebase/php-jwt	^7.2 (Composer)	Ký và xác thực chữ ký HMAC-SHA256 (HS256) theo chuẩn RFC 7519; kiểm tra thời hạn exp và claims an toàn.
Lớp truy cập CSDL	PHP Data Objects (PDO)	MySQL Driver (InnoDB)	Hỗ trợ câu lệnh chuẩn bị Prepared Statements ở tầng máy chủ (EMULATE_PREPARES = false) và quản lý giao dịch ACID.
Hệ quản trị CSDL	MySQL / MariaDB	8.0+ / 10.4+	Đảm bảo toàn vẹn dữ liệu qua ràng buộc khóa ngoại (FK ON DELETE RESTRICT) và hỗ trợ khóa dòng SELECT ... FOR UPDATE.
Giao diện Frontend	HTML5, CSS3, Vanilla JS	Fetch API, ES6+	Tách biệt 100% khỏi mã PHP; giao tiếp bất đồng bộ qua REST API, cập nhật DOM động mà không tải lại trang web.
Tài liệu & Kiểm thử	OpenAPI Specification, PHP CLI	OpenAPI 3.0.3, Postman v2.1	Tài liệu hóa hợp đồng giao tiếp độc lập nền tảng và tự động hóa quy trình kiểm thử qua 65 ca test tích hợp.

(Nguồn: Nhóm tác giả tự tổng hợp)

3.2.3 Sơ đồ tuần tự các luồng nghiệp vụ trọng tâm

Hai luồng nghiệp vụ cốt lõi của hệ thống được mô hình hóa bằng sơ đồ tuần tự UML: Quy trình đăng nhập cấp phát Bearer JWT (Hình 3.3) và Quy trình tạo phiếu mượn sách có giao dịch ACID kết hợp khóa dòng dữ liệu (Hình 3.4).

[VỊ TRÍ CHÈN HÌNH 3.3: SƠ ĐỒ TUẦN TỰ (SEQUENCE DIAGRAM) LUỒNG ĐĂNG NHẬP CẤP PHÁT JWT VÀ XÁC THỰC YÊU CẦU TRUY CẬP TÀI NGUYÊN BẢO VỆ]

Hướng dẫn: Vẽ sơ đồ Sequence Diagram UML gồm các Lifeline: Client (api.js), Front Controller (api/index.php), Auth Middleware (api/common/auth.php), Auth Module, và MySQL DB. Minh họa: POST /api/auth/login -> xác minh mật khẩu Bcrypt -> ký JWT HS256 (TTL 900s) -> trả access_token -> Client lưu LocalStorage -> Gửi request bảo vệ kèm Header 'Authorization: Bearer <token>' -> Giải mã JWT và xác minh trạng thái is_active

Hình 3.3: Sơ đồ tuần tự (Sequence Diagram) luồng đăng nhập cấp phát JWT và xác thực yêu cầu truy cập tài nguyên bảo vệ

(Nguồn: Nhóm tác giả tự thiết kế)

[VỊ TRÍ CHÈN HÌNH 3.4: SƠ ĐỒ TUẦN TỰ (SEQUENCE DIAGRAM) LUỒNG TẠO PHIẾU MƯỢN SÁCH VỚI GIAO DỊCH NGUYÊN TỬ (ACID TRANSACTION) VÀ KHÓA DÒNG DỮ LIỆU]

Hướng dẫn: Vẽ sơ đồ Sequence Diagram UML cho POST /api/loans gồm các Lifeline: Reader Client, api/index.php, auth.php, loans/create.php, và MySQL InnoDB. Minh họa Transaction: beginTransaction() -> kiểm tra user is_active -> đếm phiếu active (< 3) -> kiểm tra trùng sách -> SELECT books FOR UPDATE (available_quantity > 0) -> INSERT loans -> UPDATE books giảm tồn kho -> commit() -> trả 201 Created

Hình 3.4: Sơ đồ tuần tự (Sequence Diagram) luồng tạo phiếu mượn sách với giao dịch nguyên tử (ACID Transaction) và khóa dòng dữ liệu

(Nguồn: Nhóm tác giả tự thiết kế)

3.3 Thiết kế cơ sở dữ liệu, hợp đồng REST API và áp dụng lý thuyết vào thực nghiệm

3.3.1 Thiết kế lược đồ cơ sở dữ liệu quan hệ và các ràng buộc toàn vẹn

Cơ sở dữ liệu library_rest được thiết kế ở dạng chuẩn 3NF gồm 4 bảng thực thể: users, categories, books và loans (Hình 3.5 và Bảng 3.3). Toàn bộ khóa ngoại trên bảng books và loans đều áp dụng chính sách ON DELETE RESTRICT nhằm ngăn chặn việc xóa nhầm dữ liệu đang có liên kết nghiệp vụ.

[VỊ TRÍ CHÈN HÌNH 3.5: SƠ ĐỒ QUAN HỆ THỰC THỂ (ERD) CỦA CƠ SỞ DỮ LIỆU LIBRARY_REST TRONG HỆ THỐNG READOWL LIBRARY]

Hướng dẫn: Vẽ sơ đồ ERD gồm 4 bảng: users (id PK, full_name, email UNIQUE, password_hash, role, is_active); categories (id PK, name UNIQUE, description); books (id PK, category_id FK, title, author, isbn UNIQUE, total_quantity, available_quantity); loans (id PK, user_id FK, book_id FK, loan_date, due_date, return_date, status). Thể hiện quan hệ 1-N từ categories sang books, 1-N từ users sang loans, và 1-N từ books sang loans

Hình 3.5: Sơ đồ quan hệ thực thể (ERD) của cơ sở dữ liệu library_rest trong hệ thống ReadOwl Library

(Nguồn: Nhóm tác giả trích xuất từ database/schema.sql)

Bảng 3.3: Đặc tả cấu trúc các bảng dữ liệu, khóa ngoại và chỉ mục tối ưu hóa trong cơ sở dữ liệu library_rest

Tên bảng	Khóa chính (PK)	Khóa ngoại (FK) & Ràng buộc	Chỉ mục tối ưu hóa truy vấn (Indexes)
users	id (INT UNSIGNED AUTO_INCREMENT)	uq_users_email (email UNIQUE); role IN ('admin', 'reader'); is_active IN (0, 1)	idx_users_role, idx_users_is_active
categories	id (INT UNSIGNED AUTO_INCREMENT)	uq_categories_name (name UNIQUE)	B-Tree index trên khóa chính và cột name
books	id (INT UNSIGNED AUTO_INCREMENT)	fk_books_category -> categories(id) ON DELETE RESTRICT; uq_books_isbn (isbn UNIQUE); CHECK (available_quantity <= total_quantity)	idx_books_category_id, idx_books_title, idx_books_author
loans	id (INT UNSIGNED AUTO_INCREMENT)	fk_loans_user -> users(id) ON DELETE RESTRICT; fk_loans_book -> books(id) ON DELETE RESTRICT; status IN ('borrowed','returned','overdue','cancelled')	idx_loans_user_id, idx_loans_book_id, idx_loans_status, idx_loans_user_status (user_id, status)

(Nguồn: Nhóm tác giả trích xuất từ database/schema.sql)

3.3.2 Thiết kế không gian tài nguyên URI, máy trạng thái phiếu mượn và hợp đồng JSON

Vòng đời của phiếu mượn tuân thủ Máy trạng thái hữu hạn (Finite State Machine – FSM) được mô tả tại Hình 3.6: chỉ có các bước chuyển trạng thái hợp lệ

mới được thực thi và tự động hoàn trả số lượng tồn kho của sách khi phiếu mượn kết thúc (returned hoặc cancelled).

[VỊ TRÍ CHÈN HÌNH 3.6: SƠ ĐỒ MÁY TRẠNG THÁI HỮU HẠN (FINITE STATE MACHINE) CỦA TÀI NGUYÊN PHIẾU MƯỢN (LOAN) VÀ CƠ CHẾ ĐỒNG BỘ TỒN KHO SÁCH]

Hướng dẫn: Vẽ sơ đồ State Diagram gồm 4 trạng thái: borrowed (khởi tạo, tồn kho giảm 1), overdue (quá hạn), returned (đã trả, tồn kho tăng 1) và cancelled (đã hủy, tồn kho tăng 1). Cung chuyển hợp lệ: borrowed -> overdue, borrowed -> returned, borrowed -> cancelled, overdue -> returned. Chặn mọi phép chuyển từ returned hoặc cancelled

Hình 3.6: Sơ đồ máy trạng thái hữu hạn (Finite State Machine) của tài nguyên Phiếu mượn (Loan) và cơ chế đồng bộ tồn kho sách

(Nguồn: Nhóm tác giả tự thiết kế từ logic update.php)

Toàn bộ 25 điểm cuối RESTful API của hệ thống được tổng hợp chi tiết tại Bảng 3.4.

Bảng 3.4: Bản đồ toàn bộ 25 điểm cuối RESTful API (API Endpoints Mapping) của hệ thống ReadOwl Library

Phân hệ	Phương thức & Đường dẫn URI	Quyền hạn	Chức năng nghiệp vụ chính	Mã HTTP phản hồi
Health	GET /api/health	Public	Kiểm tra kết nối CSDL và tình trạng hoạt động của API	200, 500
Auth	POST /api/auth/register	Public	Đăng ký tài khoản độc giả mới và tự động cấp JWT	201, 409, 422
Auth	POST /api/auth/login	Public	Xác thực email/mật khẩu và cấp mã Bearer JWT HS256	200, 401, 403, 422
Auth	POST /api/auth/logout	Reader / Admin	Xác nhận đăng xuất phía máy chủ, máy khách xóa JWT	200, 401
Auth	GET, PUT /api/auth/me	Reader / Admin	Xem hoặc cập nhật hồ sơ cá nhân / đổi mật khẩu	200, 401, 422
Users	GET /api/users	Admin	Liệt kê danh sách người dùng kèm tìm kiếm, lọc và phân trang	200, 401, 403
Users	GET /api/users/{id}	Chính chủ / Admin	Xem chi tiết người dùng kèm thống kê số phiếu mượn	200, 401, 403, 404

Phân hệ	Phương thức & Đường dẫn URI	Quyền hạn	Chức năng nghiệp vụ chính	Mã HTTP phản hồi
Users	PUT, DELETE /api/users/{id}	Chính chủ / Admin	Cập nhật tài khoản hoặc xóa người dùng chưa có phiếu mượn	200, 403, 404, 409, 422
Categories	GET /api/categories, GET /api/categories/{id}	Public	Liệt kê và xem chi tiết thể loại kèm số đầu sách	200, 404
Categories	POST, PUT, DELETE /api/categories/{id}	Admin	Thêm, sửa hoặc xóa thể loại rỗng (chặn xóa nếu đang có sách)	200, 201, 401, 403, 404, 409, 422
Books	GET /api/books, GET /api/books/{id}	Public	Tìm kiếm từ khóa, lọc thể loại/tồn kho, phân trang và xem chi tiết sách	200, 404, 422
Books	POST, PUT, DELETE /api/books/{id}	Admin	Thêm sách mới, cập nhật thông tin/tồn kho, xóa sách chưa có phiếu mượn	200, 201, 401, 403, 404, 409, 422
Loans	GET /api/loans, GET /api/loans/{id}	Reader / Admin	Xem danh sách phiếu mượn (độc giả chỉ xem phiếu của mình) và chi tiết	200, 401, 403, 404
Loans	POST /api/loans	Reader / Admin	Tạo phiếu mượn sách (kiểm tra hạn mức <= 3, chống mượn trùng, giảm tồn kho)	201, 401, 404, 409, 422
Loans	PUT /api/loans/{id}	Reader / Admin	Cập nhật trạng thái phiếu mượn theo FSM, gia hạn và tự động hoàn trả tồn kho	200, 401, 403, 404, 409, 422
Statistics	GET /api/statistics/summary	Admin	Tổng hợp số đầu sách, bản sao, độc giả, Top sách mượn nhiều và phiếu quá hạn	200, 401, 403

(Nguồn: Nhóm tác giả tổng hợp từ mã nguồn api/modules/ và tài liệu docs/openapi.yaml)

3.3.3 Áp dụng các khái niệm lý thuyết Chương 2 vào hiện thực mã nguồn

Tính chuẩn mực của hệ thống được thể hiện qua các đoạn mã nguồn cốt lõi. Đầu tiên, cơ chế chuẩn hóa phong bì JSON đồng nhất và kiểm soát CORS tại api/common/response.php:

```
// api/common/response.php: Chuẩn hóa phản hồi JSON và mã trạng thái HTTP
function api_json_response(array $payload, int $statusCode = 200): never {
```

```
    if (!headers_sent()) {
        api_send_cors_headers();
        header('Content-Type: application/json; charset=utf-8');
        http_response_code($statusCode);
    }
    echo json_encode($payload, JSON_UNESCAPED_UNICODE |
JSON_UNESCAPED_SLASHES);
    exit;
}
function api_error(string $message, int $statusCode = 400, mixed $errors =
null): never {
    $payload = ['success' => false, 'message' => $message];
    if ($errors !== null) { $payload['errors'] = $errors; }
    api_json_response($payload, $statusCode);
}
```

Thứ hai, cơ chế xác thực Bearer JWT kết hợp kiểm tra trạng thái hoạt động của tài khoản tại api/common/auth.php:

```
// api/common/auth.php: Xác thực mã thông báo Bearer JWT HS256
function api_require_auth(): array {
    $token = api_get_bearer_token();
    if ($token === null) {
        api_error('Missing or invalid Authorization Bearer token.', 401);
    }
    try {
        $secret = (string) api_config('jwt_secret', '');
        $decoded = JWT::decode($token, new Key($secret, 'HS256'));
    } catch (ExpiredException) {
        api_error('Authentication token has expired.', 401);
    } catch (Throwable) {
        api_error('Invalid authentication token.', 401);
    }
    return api_verify_active_user_from_claims((array) $decoded);
}
```

Thứ ba, cơ chế giao dịch ACID và khóa dòng dữ liệu bi quan (Pessimistic Locking) bảo vệ số lượng sách tồn kho khi tạo phiếu mượn tại api/modules/loans/create.php:

```
// api/modules/loans/create.php: Giao dịch nguyên tử và khóa dòng tồn kho
$pdo->beginTransaction();
$bookStmt = $pdo->prepare(
    'SELECT id, title, available_quantity FROM books WHERE id = :id FOR
UPDATE'
);
```

```
$bookStmt->execute(['id' => $bookId]);
$book = $bookStmt->fetch();
if ((int) $book['available_quantity'] <= 0) {
    $pdo->rollBack();
    api_error('No available copies left for this book.', 409);
}
// Chèn phiếu mượn mới và giảm available_quantity đi 1 trước khi commit
```

Thứ tư, cơ chế khóa dòng theo thứ tự chuẩn users -> books -> loans tại api/modules/loans/update.php nhằm triệt tiêu hoàn toàn nguy cơ tranh chấp khóa (Deadlock):

```
// api/modules/loans/update.php: Khóa dòng theo thứ tự chuẩn tránh Deadlock
$pdo->beginTransaction();
// 1. Khóa bản ghi độc giả
$uStmt = $pdo->prepare('SELECT id, is_active FROM users WHERE id = :id FOR
UPDATE');
$uStmt->execute(['id' => $ownerUserId]);
// 2. Khóa bản ghi đầu sách
$bStmt = $pdo->prepare('SELECT id, available_quantity FROM books WHERE id =
:id FOR UPDATE');
$bStmt->execute(['id' => $targetBookId]);
// 3. Khóa bản ghi phiếu mượn và kiểm tra chuyển trạng thái hợp lệ
$lStmt = $pdo->prepare('SELECT id, status FROM loans WHERE id = :id FOR
UPDATE');
$lStmt->execute(['id' => $loanId]);
// Thực thi cập nhật trạng thái và commit transaction
```

Thứ năm, lớp tiện ích JavaScript Client SDK (public/assets/js/api.js) tự động gắn JWT vào tiêu đề HTTP và xử lý lỗi đồng bộ:

```
// public/assets/js/api.js: Client SDK tự động đính kèm Bearer JWT
async function apiRequest(endpoint, options = {}) {
    const token = localStorage.getItem('access_token');
    const headers = { 'Accept': 'application/json', ...(options.headers ||
{}) };
    if (options.body && !(options.body instanceof FormData)) {
        headers['Content-Type'] = 'application/json';
    }
    if (token) { headers['Authorization'] = `Bearer ${token}`; }
    const response = await fetch(`${API_BASE}${endpoint}`, { ...options,
headers });
    const data = await response.json();
    if (!response.ok) { throw { status: response.status, ...data }; }
    return data;
}
```

Bảng 3.5 tổng hợp sự ánh xạ trực tiếp giữa lý thuyết kiến trúc REST và các thành phần hiện thực mã nguồn trong dự án.

Bảng 3.5: Bảng ánh xạ giữa các nguyên lý lý thuyết RESTful Chương 2 và hiện thực mã nguồn trong dự án ReadOwl Library

Nguyên lý kiến trúc RESTful	Giải pháp kỹ thuật áp dụng trong ReadOwl Library	Tệp mã nguồn hiện thực chính
Phân tách Client – Server	Tách rời giao diện tĩnh HTML5/JS và dịch vụ PHP REST API giao tiếp qua JSON	public/*.html, public/assets/js/api.js, api/index.php
Phi trạng thái & Xác thực JWT	Không dùng Session; xác thực qua tiêu đề Authorization: Bearer <JWT> HS256 (TTL 900s)	api/common/auth.php, api/modules/auth/login.php
Giao diện đồng nhất & Mã HTTP	Chuẩn hóa phong bì JSON và trả đúng mã HTTP 200, 201, 400, 401, 403, 404, 409, 422	api/common/response.php, api/common/request.php
Phân trang và lọc dữ liệu	Hỗ trợ tham số truy vấn ?q=, ?category_id=, ?page=, ?limit= kèm meta.pagination	api/modules/books/index.php, api/modules/loans/index.php
Kiểm soát đồng thời (Concurrency)	Sử dụng PDO Transaction kết hợp SELECT ... FOR UPDATE và khóa theo thứ tự chuẩn	api/modules/loans/create.php, api/modules/loans/update.php
Tài liệu hóa hợp đồng API	Định nghĩa đầy đủ lược đồ dữ liệu, mã lỗi và BearerAuth bằng chuẩn OpenAPI 3.0.3	docs/openapi.yaml, docs/openapi-catalog.yaml

(Nguồn: Nhóm tác giả tự tổng hợp)

CHƯƠNG 4. THỰC NGHIỆM

4.1 Dữ liệu thực nghiệm

4.1.1 Cấu trúc bộ dữ liệu khởi tạo (Seed Dataset) và tài khoản kiểm thử

Để phục vụ công tác triển khai thực nghiệm, kiểm thử tích hợp tự động và đánh giá toàn diện các tình huống nghiệp vụ biên (edge cases) của hệ thống ReadOwl Library, nhóm thực hiện đã xây dựng bộ dữ liệu khởi tạo chuẩn hóa tại tệp database/seed.sql kết hợp với kịch bản đồng bộ mật khẩu tự động tại tệp scripts/init-auth.php. Thay vì sử dụng các bản ghi ngẫu nhiên thiếu tính liên kết, bộ dữ liệu thực nghiệm được thiết kế có chủ đích nhằm bao phủ đầy đủ mọi trạng thái của người dùng, thể loại, đầu sách và vòng đời phiếu mượn. Cụ thể, hệ thống được khởi tạo với 6 tài khoản người dùng (gồm 1 quản trị viên và 5 độc giả ở các trạng thái khác nhau), 6 thể loại sách (trong đó có 5 thể loại chứa sách và 1 thể loại rỗng phục vụ kiểm thử thao tác xóa DELETE), 20 đầu sách học thuật và kỹ thuật phần mềm (với tổng cộng 73 bản sao vật lý và 66 bản sao đang sẵn sàng cho mượn), cùng 15 bản ghi giao dịch mượn – trả sách đại diện cho cả 4 trạng thái trong máy trạng thái hữu hạn.

Bảng 4.1 trình bày chi tiết cấu trúc thống kê của bộ dữ liệu thực nghiệm trong cơ sở dữ liệu library_rest cùng danh sách các tài khoản mẫu đã được thiết lập sẵn để kiểm chứng từng kịch bản phân quyền và ràng buộc nghiệp vụ.

Bảng 4.1: Thống kê bộ dữ liệu khởi tạo (Seed Dataset) và danh sách tài khoản kiểm thử trong database/seed.sql

Nhóm dữ liệu / Tài khoản	Số lượng / Định danh	Thông tin cấu hình / Mật khẩu mẫu	Mục đích thiết kế trong kịch bản thực nghiệm và kiểm thử
Bảng categories	6 bản ghi (ID: 1 -> 6)	5 thể loại có sách (CNTT, Kinh tế, Văn học, Kỹ năng, Khoa học); Thể loại ID=6 rỗng	Kiểm thử lọc sách theo thể loại; kiểm thử chặn xóa thể loại đang có sách (409 Conflict) tại ID 1..5 và cho phép xóa thể loại rỗng tại ID 6.

Nhóm dữ liệu / Tài khoản	Số lượng / Định danh	Thông tin cấu hình / Mật khẩu mẫu	Mục đích thiết kế trong kịch bản thực nghiệm và kiểm thử
Bảng books	20 đầu sách (ID: 1 -> 20)	Tổng 73 bản sao (total_quantity), 66 bản sao khả dụng (available_quantity)	Kiểm thử tìm kiếm đa tiêu chí (tiêu đề, tác giả, ISBN), phân trang (page, limit), kiểm thử khóa dòng tồn kho khi mượn/trả sách.
Bảng loans	15 phiếu mượn (ID: 1 -> 15)	5 phiếu borrowed, 2 phiếu overdue, 6 phiếu returned, 2 phiếu cancelled	Kiểm chứng máy trạng thái hữu hạn (FSM), thống kê sách quá hạn và xếp hạng sách được mượn nhiều nhất (Top Borrowed Books).
Quản trị viên (Admin)	admin@example.com (ID: 1)	Mật khẩu: Admin12345 (role = 'admin', is_active = 1)	Kiểm thử toàn quyền quản trị CRUD danh mục, quản lý người dùng, duyệt trả sách, đánh dấu quá hạn và xem bảng điều khiển thống kê.
Độc giả chuẩn (Reader 1)	an@example.com (ID: 2)	Mật khẩu: ThuVien123 (role = 'reader', is_active = 1, đang mượn 2 cuốn)	Tài khoản độc giả chính để thực nghiệm đăng nhập, cập nhật hồ sơ, tạo phiếu mượn sách mới (còn 1 suất trong hạn mức 3 cuốn) và hủy phiếu.
Độc giả chạm hạn mức	anh.hoang@example.com (ID: 4)	Mật khẩu: ThuVien123 (role = 'reader', is_active = 1, đang mượn 3 cuốn)	Kiểm thử quy tắc chặn mượn sách khi độc giả đã chạm ngưỡng tối đa 3 phiếu mượn đang hoạt động (trả về mã lỗi HTTP 409 Conflict).
Độc giả chưa có giao dịch	khoi.vo@example.com (ID: 5)	Mật khẩu: ThuVien123 (role = 'reader', is_active = 1, 0 phiếu mượn)	Kiểm thử thao tác DELETE /api/users/{id} thành công đối với tài khoản chưa từng phát sinh lịch sử mượn trả sách.
Độc giả bị khóa	bao.pham@example.com (ID: 6)	Mật khẩu: ThuVien123 (role = 'reader', is_active = 0)	Kiểm thử cơ chế từ chối cấp phát JWT khi đăng nhập (HTTP 403 Forbidden) và thu hồi quyền truy cập ngay lập tức đối với tài khoản bị vô hiệu hóa.

(Nguồn: Nhóm tác giả trích xuất từ tệp database/seed.sql và scripts/init-auth.php)

4.1.2 Thiết kế ma trận kịch bản kiểm thử tích hợp tự động (Automated API Test Suites)

Nhằm đảm bảo toàn bộ 25 điểm cuối RESTful API tuân thủ tuyệt đối hợp đồng thiết kế tại Chương 3 và không phát sinh lỗi hồi quy khi hợp nhất mã nguồn giữa các thành viên, nhóm đã xây dựng hai bộ kịch bản kiểm thử tích hợp tự động chạy trực tiếp trên giao diện dòng lệnh PHP CLI, bao gồm tệp tests/run-api-tests.php (34 ca kiểm thử chuyên sâu cho phân hệ Xác thực, Người dùng và Mượn trả của Thành viên 1) và tệp tests/run-member2-tests.php (31 ca kiểm thử chuyên sâu cho phân hệ Thể loại, Đầu sách, Thống kê vàKiểm tra sức khỏe hệ thống của Thành viên 2), cùng hai bộ sưu tập kiểm thử Postman tại thư mục tests/postman/. Tổng cộng 65 ca kiểm thử tự động được tổ chức thành các nhóm kịch bản bao phủ cả luồng thành công (Happy Paths) lẫn các luồng ngoại lệ, vi phạm dữ liệu và tấn công vượt quyền (Negative & Security Paths) như tổng hợp tại Bảng 4.2.

Bảng 4.2: Ma trận 65 kịch bản kiểm thử tích hợp tự động trên hai bộ công cụ CLI Test Suites của dự án

Bộ kiểm thử (Test Suite)	Phân hệ kiểm tra	Số lượng Test Cases	Các kịch bản kiểm chứng trọng tâm và mã HTTP kỳ vọng
run-api-tests.php (Thành viên 1)	Auth Module (/api/auth/*)	11 ca kiểm thử	Đăng ký thành công (201), trùng email (409), dữ liệu rỗng (422); đăng nhập đúng (200), sai mật khẩu (401), tài khoản bị khóa (403); xem/sửa hồ sơ (200), đổi mật khẩu và đăng xuất (200).
run-api-tests.php (Thành viên 1)	Users Module (/api/users/*)	9 ca kiểm thử	Chặn Reader xem danh sách user (403); Admin lọc/phân trang user (200); Reader chỉ xem/sửa được chính mình (200/403); chặn Admin tự hạ quyền/tự khóa/tự xóa (422); chặn xóa user đã có phiếu mượn (409).

Bộ kiểm thử (Test Suite)	Phân hệ kiểm tra	Số lượng Test Cases	Các kịch bản kiểm chứng trọng tâm và mã HTTP kỳ vọng
run-api-tests.php (Thành viên 1)	Loans Module (/api/loans/*)	14 ca kiểm thử	Tạo phiếu mượn giảm available_quantity đi 1 (201); chặn mượn trùng sách đang giữ (409); chặn vượt hạn mức 3 cuốn (409); chặn mượn sách hết tồn kho (409); kiểm tra FSM chuyển trạng thái borrowed -> overdue -> returned hoàn trả tồn kho (200); chặn mở lại phiếu đã trả (409).
run-member2-tests.php (Thành viên 2)	Health & Root (/api, /api/health)	3 ca kiểm thử	Kiểm tra thông tin định tuyến gốc (200), kiểm tra kết nối PDO MySQL sống (200) và từ chối phương thức không hợp lệ (405 Method Not Allowed).
run-member2-tests.php (Thành viên 2)	Categories Module (/api/categories/*)	11 ca kiểm thử	Liệt kê và xem chi tiết thể loại công khai (200, 404); chặn người dùng chưa đăng nhập hoặc Reader thêm/sửa/xóa thể loại (401, 403); Admin thêm/sửa thể loại (201, 200), bắt lỗi trùng tên (409), chặn xóa thể loại đang chứa sách (409) và xóa thể loại rỗng (200).
run-member2-tests.php (Thành viên 2)	Books Module (/api/books/*)	13 ca kiểm thử	Tìm kiếm theo từ khóa, lọc theo category_id, lọc sách còn tồn kho available=1, kiểm tra siêu dữ liệu phân trang (200); kiểm tra tham số sai (422); Admin thêm/sửa sách (201, 200), chặn trùng ISBN (409), chặn đặt total_quantity nhỏ hơn số sách đang cho mượn (409), chặn xóa sách đã có lịch sử mượn (409).
run-member2-tests.php (Thành viên 2)	Statistics Module (/api/statistics/summary)	4 ca kiểm thử	Chặn truy cập ẩn danh hoặc Reader (401, 403); Admin truy xuất báo cáo tổng hợp và đối chiếu chéo số liệu đầu sách, bản sao, phiếu mượn với kết quả đếm trực tiếp từ cơ sở dữ liệu (200).

(Nguồn: Nhóm tác giả tổng hợp từ tệp tests/run-api-tests.php và tests/run-member2-tests.php)

4.2 Cài đặt thực nghiệm

4.2.1 Môi trường phần cứng, phần mềm và cấu trúc cây thư mục dự án

Hệ thống ReadOwl Library được cài đặt và thực nghiệm trên môi trường máy chủ cục bộ chạy hệ điều hành Windows 10/11 (64-bit) với gói phần mềm XAMPP tích hợp trình thông dịch PHP 8.2 (bật các phần mở rộng pdo_mysql, mbstring, openssl, curl), hệ quản trị cơ sở dữ liệu MySQL 8.0 / MariaDB 10.4 và trình quản lý thư viện phụ thuộc Composer 2.x. Phụ thuộc bên ngoài duy nhất của tầng Backend được khai báo trong tệp composer.json là gói firebase/php-jwt phiên bản ^7.2 dùng để ký và xác thực mã thông báo JWT chuẩn HS256.

Mã nguồn của dự án được tổ chức theo cấu trúc mô-đun hóa rõ ràng, tách biệt hoàn toàn giữa cấu hình hệ thống (config/), lược đồ dữ liệu (database/), tầng dịch vụ RESTful API (api/), giao diện người dùng tĩnh (public/), tài liệu đặc tả OpenAPI (docs/) và bộ công cụ kiểm thử tự động (tests/) như minh họa dưới đây:

```
ReadOwl-Library/
├── api/
│   ├── common/          # Tiện ích lõi: db.php, request.php, response.php,
auth.php
│   ├── modules/         # 6 phân hệ REST: auth, users, categories, books,
loans, statistics
│   └── index.php        # Bộ điều khiển trung tâm (Front Controller &
Router)
├── config/              # Cấu hình CSDL và tham số ứng dụng (config.php,
app.php)
├── database/            # Lược đồ CSDL và dữ liệu mẫu (schema.sql,
seed.sql)
├── docs/                # Đặc tả OpenAPI 3.0.3 (openapi.yaml) và tài liệu
phân hệ
├── public/              # Giao diện HTML5/CSS3/JS cho Độc giả và Cổng Admin
(public/admin/)
├── scripts/             # Kịch bản khởi tạo xác thực (init-auth.php)
├── tests/               # 65 ca kiểm thử CLI tự động và Postman Collections
└── router.php           # Bộ định tuyến cho máy chủ tích hợp PHP CLI
```

4.2.2 Quy trình triển khai hệ thống và khởi chạy dịch vụ thực nghiệm

Quy trình triển khai thực nghiệm hệ thống ReadOwl Library từ mã nguồn sạch được chuẩn hóa qua bốn bước tuần tự giúp đảm bảo khả năng tái lập 100% trên mọi

máy tính phát triển. Bước thứ nhất là cài đặt thư viện phụ thuộc JWT thông qua lệnh composer install tại thư mục gốc của dự án để khởi tạo thư mục vendor/. Bước thứ hai là nạp cấu trúc bảng và dữ liệu mẫu vào hệ quản trị MySQL thông qua hai tệp database/schema.sql và database/seed.sql. Bước thứ ba là thực thi kịch bản php scripts/init-auth.php để đồng bộ hóa chuỗi băm mật khẩu chuẩn Bcrypt cho toàn bộ tài khoản mẫu và xác nhận kết nối PDO hoạt động bình thường. Bước thứ tư là khởi chạy máy chủ phát triển tích hợp của PHP bằng lệnh php -S localhost:8000 router.php (hoặc vận hành trực tiếp qua cổng Apache của XAMPP). Kết quả khởi tạo cơ sở dữ liệu và khởi chạy máy chủ thực nghiệm được thể hiện tại Hình 4.1.

[VỊ TRÍ CHÈN HÌNH 4.1: GIAO DIỆN DÒNG LỆNH THỰC THI KỊCH BẢN KHỞI TẠO XÁC THỰC (SCRIPTS/INIT-AUTH.PHP) VÀ KHỞI CHẠY MÁY CHỦ RESTFUL API CỤC BỘ]

Hướng dẫn: Chụp màn hình cửa sổ Terminal / PowerShell khi chạy lệnh 'php scripts/init-auth.php' (hiển thị thông báo đồng bộ thành công mật khẩu Bcrypt cho 6 tài khoản mẫu và trạng thái kết nối CSDL library_rest) và lệnh 'php -S localhost:8000 router.php' đang lắng nghe yêu cầu HTTP

Hình 4.1: Giao diện dòng lệnh thực thi kịch bản khởi tạo xác thực (scripts/init-auth.php) và khởi chạy máy chủ RESTful API cục bộ

(Nguồn: Nhóm tác giả chụp màn hình thực nghiệm hệ thống)

4.3 Kết quả và đánh giá

4.3.1 Kết quả thực nghiệm giao diện ứng dụng Web tích hợp RESTful API

Sau khi khởi chạy dịch vụ, nhóm tiến hành thực nghiệm các luồng nghiệp vụ từ phía người dùng cuối thông qua giao diện trình duyệt nhằm kiểm chứng sự phối hợp bất đồng bộ giữa mô-đun public/assets/js/api.js và các điểm cuối RESTful API. Hình 4.2 minh họa trang chủ tra cứu danh mục sách dành cho khách vãng lai và độc giả (public/index.html). Khi trang web được tải, kịch bản JavaScript tự động gửi đồng thời hai yêu cầu GET /api/categories và GET /api/books để kết xuất bộ lọc thể loại và lưới thẻ sách kèm nhãn trạng thái tồn kho trực quan.

[VỊ TRÍ CHÈN HÌNH 4.2: GIAO DIỆN TRANG CHỦ THƯ VIỆN READOWL LIBRARY (PUBLIC/INDEX.HTML) HIỂN THỊ DANH MỤC SÁCH, BỘ LỌC THỂ LOẠI VÀ TÌM KIẾM THỜI GIAN THỰC]

Hướng dẫn: Chụp màn hình trình duyệt tại địa chỉ http://localhost:8000/public/index.html hiển thị giao diện ReadOwl Library với thanh tìm kiếm từ khóa, bộ lọc thể loại sách, danh sách các đầu sách kèm số lượng bản sao còn lại (available_quantity / total_quantity) và phân trang

Hình 4.2: Giao diện trang chủ thư viện ReadOwl Library (public/index.html) hiển thị danh mục sách, bộ lọc thể loại và tìm kiếm thời gian thực

(Nguồn: Nhóm tác giả chụp màn hình thực nghiệm hệ thống)

Tiếp theo, luồng đăng nhập nhận mã thông báo JWT (public/login.html) và giao diện quản lý hồ sơ cá nhân (public/profile.html) được thể hiện tại Hình 4.3. Khi đăng nhập thành công bằng tài khoản độc giả (an@example.com) hoặc quản trị viên (admin@example.com), chuỗi access_token trả về từ điểm cuối POST /api/auth/login được lưu trữ vào bộ nhớ localStorage của trình duyệt và có thể quan sát trực tiếp trong công cụ dành cho nhà phát triển (Browser DevTools).

[VỊ TRÍ CHÈN HÌNH 4.3: GIAO DIỆN ĐĂNG NHẬP NHẬN MÃ THÔNG BÁO JWT (PUBLIC/LOGIN.HTML) VÀ KIỂM TRA CHUỖI BEARER TOKEN TRONG DEVTOOLS LOCALSTORAGE]

Hướng dẫn: Chụp màn hình trang đăng nhập http://localhost:8000/public/login.html hoặc trang Hồ sơ cá nhân http://localhost:8000/public/profile.html kèm cửa sổ Chrome DevTools (tab Application -> Local Storage hoặc tab Network) hiển thị rõ chuỗi JWT Bearer Token được trả về từ API

Hình 4.3: Giao diện đăng nhập nhận mã thông báo JWT (public/login.html) và kiểm tra chuỗi Bearer Token trong DevTools LocalStorage

(Nguồn: Nhóm tác giả chụp màn hình thực nghiệm hệ thống)

Hình 4.4 minh họa giao diện chi tiết đầu sách (public/book-detail.html) và trang quản lý phiếu mượn cá nhân của độc giả (public/my-loans.html). Tại đây, độc giả có thể gửi yêu cầu mượn sách mới (POST /api/loans), theo dõi ngày hẹn trả, nhận cảnh báo trực quan nếu phiếu mượn bị quá hạn và chủ động hủy phiếu mượn đang hoạt động (PUT /api/loans/{id} với trạng thái cancelled).

[VỊ TRÍ CHÈN HÌNH 4.4: GIAO DIỆN TRANG QUẢN LÝ PHIẾU MƯỢN CÁ NHÂN CỦA ĐỘC GIẢ (PUBLIC/MY-LOANS.HTML) VÀ CHỨC NĂNG ĐĂNG KÝ MƯỢN / HỦY PHIẾU MƯỢN]

Hướng dẫn: Chụp màn hình trang Phiếu mượn của tôi tại địa chỉ http://localhost:8000/public/my-loans.html khi đăng nhập tài khoản độc giả an@example.com, hiển thị danh sách các sách đang mượn, ngày mượn, hạn trả, trạng thái phiếu và nút thao tác Hủy phiếu / Mượn sách mới

Hình 4.4: Giao diện trang quản lý phiếu mượn cá nhân của độc giả (public/my-loans.html) và chức năng đăng ký mượn / hủy phiếu mượn

(Nguồn: Nhóm tác giả chụp màn hình thực nghiệm hệ thống)

Đối với vai trò Quản trị viên, Hình 4.5 trình bày giao diện Bảng điều khiển thống kê toàn thư viện (public/admin/dashboard.html), nơi dữ liệu tổng hợp từ điểm cuối GET /api/statistics/summary được trực quan hóa thành các thẻ chỉ số (tổng số đầu sách, bản sao đang cho mượn, số phiếu quá hạn, số độc giả hoạt động), bảng xếp hạng các đầu sách được mượn nhiều nhất và danh sách cảnh báo các phiếu mượn quá hạn cần thu hồi.

[VỊ TRÍ CHÈN HÌNH 4.5: GIAO DIỆN BẢNG ĐIỀU KHIỂN THỐNG KÊ DÀNH CHO QUẢN TRỊ VIÊN (PUBLIC/ADMIN/DASHBOARD.HTML) KẾT XUẤT DỮ LIỆU TỪ GET /API/STATISTICS/SUMMARY]

Hướng dẫn: Chụp màn hình trang Quản trị Tổng quan tại địa chỉ http://localhost:8000/public/admin/dashboard.html khi đăng nhập tài khoản admin@example.com, hiển thị các thẻ thống kê tổng quan, bảng Top sách được mượn nhiều nhất và danh sách các phiếu mượn đang quá hạn (overdue)

Hình 4.5: Giao diện Bảng điều khiển thống kê dành cho Quản trị viên (public/admin/dashboard.html) kết xuất dữ liệu từ GET /api/statistics/summary

(Nguồn: Nhóm tác giả chụp màn hình thực nghiệm hệ thống)

Hình 4.6 minh họa các giao diện quản trị danh mục sách (public/admin/books.html) và điều phối phiếu mượn trả toàn hệ thống (public/admin/loans.html), nơi quản trị viên thực hiện thêm đầu sách mới, cập nhật tồn kho và xác nhận thu hồi sách trả từ độc giả.

[VỊ TRÍ CHÈN HÌNH 4.6: GIAO DIỆN QUẢN TRỊ KHO SÁCH (PUBLIC/ADMIN/BOOKS.HTML) VÀ ĐIỀU PHỐI PHIẾU MƯỢN TRẢ (PUBLIC/ADMIN/LOANS.HTML) CỦA QUẢN TRỊ VIÊN]

Hướng dẫn: Chụp màn hình trang Quản lý sách (http://localhost:8000/public/admin/books.html) hoặc Quản lý mượn trả (http://localhost:8000/public/admin/loans.html) với biểu mẫu thêm/cập nhật sách và các nút chuyển trạng thái phiếu mượn (Đánh dấu đã trả, Đánh dấu quá hạn, Gia hạn)

Hình 4.6: Giao diện Quản trị kho sách (public/admin/books.html) và Điều phối phiếu mượn trả (public/admin/loans.html) của Quản trị viên

(Nguồn: Nhóm tác giả chụp màn hình thực nghiệm hệ thống)

4.3.2 Kết quả kiểm thử tự động, kiểm thử Postman và đánh giá hiệu năng

Để đánh giá khách quan độ tin cậy của tầng dịch vụ RESTful API, nhóm đã thực thi cả hai bộ kịch bản kiểm thử tích hợp tự động trên dòng lệnh (php tests/run-api-tests.php và php tests/run-member2-tests.php) cũng như chạy bộ sưu tập kiểm thử trên công cụ Postman (tests/postman/library-rest-full.postman_collection.json). Kết quả thực thi trên dòng lệnh cho thấy toàn bộ 34/34 ca kiểm thử của Thành viên 1 và 31/31 ca kiểm thử của Thành viên 2 (tổng cộng 65/65 ca kiểm thử, đạt tỷ lệ 100% PASS) đều vượt qua thành công mà không phát sinh bất kỳ lỗi sai lệch trạng thái HTTP hay sai lệch số lượng tồn kho nào (Hình 4.7 và Hình 4.8).

[VỊ TRÍ CHÈN HÌNH 4.7: KẾT QUẢ THỰC THI THÀNH CÔNG 100% HAI BỘ KỊCH BẢN KIỂM THỬ TỰ ĐỘNG (34/34 TESTS TRONG RUN-API-TESTS.PHP VÀ 31/31 TESTS TRONG RUN-MEMBER2-TESTS.PHP)]

Hướng dẫn: Chụp màn hình cửa sổ Terminal khi chạy hai lệnh 'php tests/run-api-tests.php' (hiển thị 34/34 passed) và 'php tests/run-member2-tests.php' (hiển thị 31/31 passed) với danh sách các bài kiểm tra được đánh dấu PASS

Hình 4.7: Kết quả thực thi thành công 100% hai bộ kịch bản kiểm thử tự động (34/34 tests trong run-api-tests.php và 31/31 tests trong run-member2-tests.php)

(Nguồn: Nhóm tác giả chụp màn hình kết quả kiểm thử tự động)

[VỊ TRÍ CHÈN HÌNH 4.8: KẾT QUẢ KIỂM THỬ CÁC ĐIỂM CUỐI RESTFUL API VÀ KIỂM TRA CẤU TRÚC PHẢN HỒI JSON TRÊN CÔNG CỤ POSTMAN]

Hướng dẫn: Chụp màn hình ứng dụng Postman khi nạp tệp tests/postman/library-rest-full.postman_collection.json và thực thi các request mẫu (ví dụ POST /api/auth/login, GET /api/books, POST /api/loans) hiển thị mã trạng thái HTTP 200/201, thời gian phản hồi (ms) và cấu trúc JSON trả về

Hình 4.8: Kết quả kiểm thử các điểm cuối RESTful API và kiểm tra cấu trúc phản hồi JSON trên công cụ Postman

(Nguồn: Nhóm tác giả chụp màn hình thực nghiệm trên Postman)

Bảng 4.3 tổng hợp kết quả kiểm thử nghiệm thu theo từng phân hệ và ghi nhận độ trễ phản hồi thực tế khi đo đạc trên môi trường máy chủ cục bộ.

Bảng 4.3: Tổng hợp kết quả thực thi kiểm thử tích hợp và đo đạc thời gian phản hồi các điểm cuối RESTful API

Phân hệ kiểm thử	Điểm cuối đại diện	Số ca kiểm thử (Pass / Total)	Mã HTTP kiểm chứng	Thời gian phản hồi trung bình (ms)
Xác thực (Auth)	POST /api/auth/login, POST /api/auth/register	11 / 11 (100% PASS)	200, 201, 401, 403, 409, 422	[CẦN BỔ SUNG: Ghi số ms đo thực tế trên Postman của nhóm, ví dụ ~45 ms do có băm Bcrypt]
Người dùng (Users)	GET /api/users, PUT/DELETE /api/users/{id}	9 / 9 (100% PASS)	200, 401, 403, 404, 409, 422	[CẦN BỔ SUNG: Ghi số ms đo thực tế trên Postman của nhóm, ví dụ ~12 ms]
Thể loại (Categories)	GET, POST, PUT, DELETE /api/categories	11 / 11 (100% PASS)	200, 201, 401, 403, 404, 409, 422	[CẦN BỔ SUNG: Ghi số ms đo thực tế trên Postman của nhóm, ví dụ ~10 ms]
Đầu sách (Books)	GET /api/books?q=...&page=1, POST/PUT/DELETE	13 / 13 (100% PASS)	200, 201, 401, 403, 404, 409, 422	[CẦN BỔ SUNG: Ghi số ms đo thực tế trên Postman của nhóm, ví dụ ~15 ms]
Mượn – Trả (Loans)	POST /api/loans, PUT /api/loans/{id}	14 / 14 (100% PASS)	200, 201, 401, 403, 404, 409, 422	[CẦN BỔ SUNG: Ghi số ms đo thực tế trên Postman của nhóm, ví dụ ~18 ms (có Transaction)]

Phân hệ kiểm thử	Điểm cuối đại diện	Số ca kiểm thử (Pass / Total)	Mã HTTP kiểm chứng	Thời gian phản hồi trung bình (ms)
Thống kê & Sức khỏe	GET /api/statistics/summary, GET /api/health	7 / 7 (100% PASS)	200, 401, 403, 405	[CẦN BỔ SUNG: Ghi số ms đo thực tế trên Postman của nhóm, ví dụ ~14 ms]

(Nguồn: Nhóm tác giả tổng hợp từ kết quả chạy CLI Test Suites và Postman)

4.4 Thách thức gặp phải và cách giải quyết

4.4.1 Các thách thức kỹ thuật phát sinh trong quá trình phát triển và tích hợp

Trong quá trình thiết kế và hiện thực hóa hệ thống ReadOwl Library bằng ngôn ngữ PHP thuần, nhóm đã đối mặt với bốn thách thức kỹ thuật lớn. Thách thức thứ nhất là hiện tượng máy chủ Apache trên môi trường Windows/XAMPP đôi khi tự động loại bỏ tiêu đề Authorization khỏi biến siêu toàn cục $_SERVER trước khi chuyển giao cho kịch bản PHP, khiến mô-đun xác thực không đọc được chuỗi Bearer JWT dù trình duyệt đã gửi đi đầy đủ. Thách thức thứ hai là sự khác biệt cơ chế định tuyến đường dẫn ảo (/api/*) giữa máy chủ phát triển tích hợp PHP CLI (php -S) và máy chủ Apache sử dụng mô-đun mod_rewrite. Thách thức thứ ba là nguy cơ sai lệch số lượng bản sao sách khả dụng (available_quantity) khi thao tác tạo phiếu mượn hoặc cập nhật trạng thái trả sách gặp lỗi giữa chừng hoặc khi quản trị viên chỉnh sửa tổng số lượng sách (total_quantity) nhỏ hơn số lượng bản sao đang được độc giả mượn. Thách thức thứ tư là bài toán thu hồi quyền truy cập tức thời đối với tài khoản độc giả bị quản trị viên khóa (is_active = 0) trong bối cảnh mã thông báo JWT phi trạng thái vẫn còn thời hạn hiệu lực.

4.4.2 Giải pháp kỹ thuật khắc phục và bài học kinh nghiệm thực tiễn

Để giải quyết triệt để bốn thách thức nêu trên, nhóm đã nghiên cứu và triển khai các giải pháp kỹ thuật chuyên sâu trong mã nguồn dự án như tổng hợp tại Bảng 4.4. Cụ thể, đối với vấn đề mất tiêu đề xác thực trên Apache, hàm api_get_authorization_header() trong tệp api/common/request.php được thiết kế theo

cơ chế dự phòng đa tầng, lần lượt kiểm tra từ $_SERVER['HTTP_AUTHORIZATION'], $_SERVER['REDIRECT_HTTP_AUTHORIZATION'] cho đến hàm apache_request_headers() và getallheaders(). Đối với bài toán định tuyến đa môi trường, nhóm xây dựng song song tệp .htaccess cho Apache và tệp router.php cho PHP CLI cùng trỏ về điểm vào duy nhất api/index.php. Đặc biệt, việc kết hợp giao dịch PDO với khóa dòng SELECT ... FOR UPDATE và công thức kiểm tra bất biến tồn kho khi cập nhật sách đã giúp loại bỏ hoàn toàn các lỗi bất nhất dữ liệu.

Bảng 4.4: Tổng hợp các thách thức kỹ thuật gặp phải trong quá trình thực nghiệm và giải pháp khắc phục trong mã nguồn

STT	Thách thức kỹ thuật gặp phải	Nguyên nhân gốc rễ (Root Cause)	Giải pháp kỹ thuật đã triển khai trong ReadOwl Library
1	Mất tiêu đề Authorization: Bearer <JWT> trên môi trường Apache/XAMPP	Cấu hình bảo mật mặc định của Apache CGI/FastCGI không ánh xạ trực tiếp header Authorization vào $_SERVER['HTTP_AUTHORIZATION']	Xây dựng hàm api_get_authorization_header() trong api/common/request.php đọc dự phòng qua 4 tầng: HTTP_AUTHORIZATION, REDIRECT_HTTP_AUTHORIZATION, apache_request_headers() và getallheaders().
2	Khác biệt cơ chế định tuyến URI (/api/*) giữa Apache và PHP Built-in Server	Máy chủ tích hợp 'php -S' không đọc tệp cấu hình .htaccess của Apache nên trả lỗi 404 cho các đường dẫn ảo không có tệp vật lý	Xây dựng đồng thời tệp .htaccess (dùng RewriteRule cho Apache) và tệp router.php (dùng cho PHP CLI) cùng chuyển tiếp mọi URI có tiền tố /api/* về Front Controller api/index.php.
3	Nguy cơ âm tồn kho hoặc lệch số lượng sách khi mượn/trả và khi Admin sửa tổng số lượng sách	Thao tác cập nhật 2 bảng loans và books không nguyên tử, hoặc Admin giảm total_quantity xuống thấp hơn số sách đang cho mượn	Bọc toàn bộ thao tác mượn/trả trong PDO Transaction kèm SELECT ... FOR UPDATE; khi sửa sách tại api/modules/books/update.php, bắt buộc kiểm tra newTotal >= borrowedCount (total - available) trước khi cho phép lưu.

STT	Thách thức kỹ thuật gặp phải	Nguyên nhân gốc rễ (Root Cause)	Giải pháp kỹ thuật đã triển khai trong ReadOwl Library
4	Độc giả bị khóa tài khoản (is_active = 0) nhưng vẫn còn giữ JWT chưa hết hạn (900 giây)	Bản chất phi trạng thái (Stateless) của JWT khiến chữ ký số vẫn hợp lệ cho đến thời điểm exp	Cài đặt hàm api_verify_active_user_from_claims() trong api/common/auth.php: sau khi xác minh chữ ký JWT thành công, đối chiếu nhanh khóa chính sub trên bảng users để chặn ngay bằng mã 403 nếu is_active = 0.

(Nguồn: Nhóm tác giả tự đúc kết từ quá trình phát triển và gỡ lỗi hệ thống)

CHƯƠNG 5. KẾT LUẬN

5.1 Kết luận

Báo cáo giữa kỳ đã hoàn thành trọn vẹn hai mục tiêu nghiên cứu lý thuyết và phát triển thực nghiệm đặt ra ban đầu đối với chủ đề Dịch vụ Web RESTful (RESTful Web Services). Về mặt lý thuyết, báo cáo đã hệ thống hóa sâu sắc lịch sử hình thành từ luận án tiến sĩ của Roy Thomas Fielding, sáu ràng buộc kiến trúc nền tảng (Phân tách Client – Server, Phi trạng thái, Khả năng lưu đệm, Giao diện đồng nhất, Hệ thống phân lớp và Mã theo yêu cầu), mô hình trưởng thành Richardson 4 cấp độ, đặc tả kỹ thuật giao thức HTTP/1.1 theo chuẩn RFC 9110, cơ chế bảo mật bằng mã thông báo JSON Web Token (RFC 7519), chuẩn tài liệu hóa OpenAPI 3.0.3 cũng như phân tích so sánh đa chiều giữa REST với SOAP, GraphQL và gRPC.

Về mặt thực nghiệm, nhóm đã vận dụng trực tiếp các nguyên lý và thực tiễn tốt nhất đó để phân tích, thiết kế và xây dựng hoàn chỉnh hệ thống quản lý thư viện mượn – trả sách trực tuyến ReadOwl Library. Hệ thống cung cấp 25 điểm cuối RESTful API đạt cấp độ 2 trên mô hình trưởng thành Richardson, được tổ chức thành 6 phân hệ nghiệp vụ rõ ràng (auth, users, categories, books, loans, statistics) trên nền tảng PHP thuần kết hợp cơ sở dữ liệu quan hệ MySQL/MariaDB. Các điểm sáng kỹ thuật của hệ thống bao gồm: cơ chế xác thực phi trạng thái bằng JWT HS256 kết hợp kiểm soát truy cập phân quyền RBAC; bảo vệ tính toàn vẹn tồn kho tuyệt đối bằng giao dịch nguyên tử PDO và khóa dòng dữ liệu (SELECT ... FOR UPDATE); kiểm soát vòng đời phiếu mượn bằng máy trạng thái hữu hạn (FSM); chuẩn hóa hợp đồng giao tiếp bằng đặc tả OpenAPI 3.0.3; và đặc biệt là việc kiểm chứng độ tin cậy thông qua 65 kịch bản kiểm thử tích hợp tự động đạt tỷ lệ thành công 100%.

5.2 Hướng phát triển

Mặc dù hệ thống ReadOwl Library đã đáp ứng đầy đủ các yêu cầu nghiệp vụ cốt lõi và tiêu chí kỹ thuật của đồ án giữa kỳ, kiến trúc hiện tại vẫn có thể được mở rộng và nâng cấp sâu hơn trong các giai đoạn tiếp theo nhằm tiệm cận môi trường vận hành quy mô công nghiệp. Trước hết, nhóm định hướng nâng cấp các điểm cuối

API từ Cấp độ 2 lên Cấp độ 3 hoàn chỉnh trên mô hình trưởng thành Richardson thông qua việc nhúng các siêu liên kết điều hướng động (HATEOAS) vào trong từng bản ghi sách và phiếu mượn. Thứ hai, để tăng cường bảo mật và trải nghiệm người dùng, cơ chế xác thực sẽ được bổ sung cặp mã thông báo làm mới (Refresh Token) lưu trong HttpOnly Cookie kết hợp với danh sách thu hồi thẻ trên bộ nhớ đệm Redis.

Bảng 5.1 tổng hợp lộ trình các hướng phát triển và nâng cấp kỹ thuật dự kiến cho hệ thống ReadOwl Library trong tương lai.

Bảng 5.1: Lộ trình các hướng phát triển mở rộng chức năng và nâng cấp kiến trúc cho hệ thống ReadOwl Library

Hạng mục nâng cấp	Hiện trạng trong phiên bản hiện tại	Giải pháp kỹ thuật hướng tới trong tương lai	Lợi ích mang lại cho hệ thống
Tiêu chuẩn HATEOAS (Richardson Level 3)	Đạt hoàn chỉnh Cấp độ 2 (URI danh từ + HTTP Verbs + HTTP Status Codes chuẩn)	Bổ sung trường _links (theo chuẩn HAL hoặc JSON:API) vào phản hồi của sách và phiếu mượn tùy theo trạng thái FSM hiện tại	Giúp máy khách tự động khám phá các hành động hợp lệ tiếp theo mà không cần lập trình cứng quy tắc xây dựng đường dẫn URI.
Cơ chế Refresh Token & Danh sách thu hồi Redis	Sử dụng Access Token JWT đơn lẻ có thời hạn 900 giây lưu trong localStorage	Triển khai cặp Access Token ngắn hạn (5 phút) kèm Refresh Token xoay vòng (Rotation) lưu trong HttpOnly Cookie và Redis Blacklist	Chống triệt để nguy cơ đánh cắp thẻ qua lỗi XSS và hỗ trợ thu hồi phiên đăng nhập tức thời mà không cần truy vấn bảng users.
Giới hạn tần suất & Bộ nhớ đệm (Rate Limiting & Caching)	Chưa giới hạn số lượng yêu cầu mỗi phút và chưa gắn thẻ ETag tự động	Tích hợp tầng kiểm soát lưu lượng (Token Bucket) trên Redis kèm tiêu đề ETag / Cache-Control cho danh mục sách và thể loại	Ngăn chặn tấn công vét cạn mật khẩu (Brute-force), chống quá tải máy chủ và giảm tới 70% truy vấn đọc lặp lại xuống MySQL.
Đóng gói Docker & Tự động hóa CI/CD	Cài đặt trên môi trường XAMPP / PHP CLI cục bộ và chạy kịch bản kiểm thử thủ công	Xây dựng tệp Dockerfile, docker-compose.yml (PHP-FPM + Nginx + MySQL) và quy trình GitHub Actions tự động chạy 65 tests khi đẩy mã	Đảm bảo tính đồng nhất môi trường triển khai đám mây và kiểm soát chất lượng mã nguồn liên tục (Continuous Integration).

(Nguồn: Nhóm tác giả tự đề xuất)

TÀI LIỆU THAM KHẢO

Tiếng Việt

Bộ Thông tin và Truyền thông. (2019). Thông tư số 13/2019/TT-BTTTT quy định chuẩn kỹ thuật về kết nối, tích hợp và chia sẻ dữ liệu trên nền tảng tích hợp, chia sẻ dữ liệu. Hà Nội.

Nguyễn, T. D. (2026). Bài giảng môn Lập trình Web và Ứng dụng (Mã môn học: 503073). Khoa Công nghệ Thông tin, Trường Đại học Tôn Đức Thắng.

Tiếng Anh

Berners-Lee, T., Fielding, R., & Masinter, L. (2005). Uniform Resource Identifier (URI): Generic syntax (RFC 3986). Internet Engineering Task Force (IETF). https://doi.org/10.17487/RFC3986

Bray, T. (Ed.). (2017). The JavaScript Object Notation (JSON) data interchange format (RFC 8259). Internet Engineering Task Force (IETF). https://doi.org/10.17487/RFC8259

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Doctoral dissertation, University of California, Irvine]. UCI Campus Repository.

Fielding, R. T., & Taylor, R. N. (2002). Principled design of the modern Web architecture. ACM Transactions on Internet Technology (TOIT), 2(2), 115–150. https://doi.org/10.1145/514183.514185

Fielding, R. T., Nottingham, M., & Reschke, J. (Eds.). (2022a). HTTP semantics (RFC 9110). Internet Engineering Task Force (IETF). https://doi.org/10.17487/RFC9110

Fielding, R. T., Nottingham, M., & Reschke, J. (Eds.). (2022b). HTTP caching (RFC 9111). Internet Engineering Task Force (IETF). https://doi.org/10.17487/RFC9111

Fowler, M. (2010, March 18). Richardson Maturity Model: Steps toward the glory of REST. MartinFowler.com. https://martinfowler.com/articles/richardsonMaturityModel.html

Hardt, D. (Ed.). (2012). The OAuth 2.0 authorization framework (RFC 6749). Internet Engineering Task Force (IETF). https://doi.org/10.17487/RFC6749

Jones, M., & Hildebrand, J. (2012). Concise binary object representation and JSON: Envelope and validation patterns (RFC 6838). Internet Engineering Task Force (IETF).

Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT) (RFC 7519). Internet Engineering Task Force (IETF). https://doi.org/10.17487/RFC7519

Masse, M. (2011). REST API design rulebook: Designing consistent RESTful Web service interfaces. O'Reilly Media.

Newman, S. (2021). Building microservices: Designing fine-grained systems (2nd ed.). O'Reilly Media.

OpenAPI Initiative. (2021). OpenAPI Specification v3.0.3. Linux Foundation. https://spec.openapis.org/oas/v3.0.3

OWASP Foundation. (2023). OWASP API Security Top 10 – 2023 edition. Open Worldwide Application Security Project. https://owasp.org/API-Security/

Pautasso, C., Zimmermann, O., & Leymann, F. (2008). RESTful web services vs. "big" web services: Making the right architectural decision. Proceedings of the 17th International Conference on World Wide Web (WWW '08), 805–814. https://doi.org/10.1145/1367497.1367606

Richardson, L., & Amundsen, M. (2013). RESTful Web APIs: Services for a changing world. O'Reilly Media.

PHỤ LỤC

Phụ lục A: Bảng phân công nhiệm vụ, mức độ hoàn thành và tự đánh giá của nhóm

Dự án nghiên cứu lý thuyết Dịch vụ Web RESTful và xây dựng hệ thống quản lý thư viện ReadOwl Library được thực hiện bởi nhóm gồm 02 thành viên. Quá trình phân chia công việc được hoạch định cân bằng từ khâu thiết kế cơ sở dữ liệu, phát triển các mô-đun REST API, xây dựng giao diện người dùng, viết đặc tả OpenAPI 3.0.3, lập trình bộ kiểm thử tự động cho đến biên soạn báo cáo học thuật. Bảng dưới đây trình bày chi tiết các hạng mục công việc, kết quả đầu ra và tỷ lệ đóng góp của từng thành viên.

MSSV	Họ và tên	Vai trò	Chi tiết công việc phụ trách (Mã nguồn & Báo cáo)	Mức độ hoàn thành	Tỷ lệ đóng góp
52400065	Trần Hồ Duy Ân	Nhóm trưởng (Thành viên 1)	- Xây dựng mô-đun tiện ích xác thực JWT HS256 & RBAC (api/common/auth.php) và kịch bản khởi tạo mật khẩu Bcrypt (scripts/init-auth.php). - Phát triển 3 phân hệ RESTful API: Auth (/api/auth/* - 5 endpoints), Users (/api/users/* - 4 endpoints) và Loans (/api/loans/* - 4 endpoints kèm ACID Transaction, khóa dòng FOR UPDATE và máy trạng thái FSM). - Xây dựng 6 giao diện Frontend: Đăng nhập (login.html), Đăng ký (register.html), Hồ sơ (profile.html), Phiếu mượn cá nhân (my-loans.html), Quản trị người dùng (admin/users.html) và Quản trị mượn trả (admin/loans.html). - Viết đặc tả docs/openapi-auth-loans.yaml, bộ kiểm thử tự động tests/run-api-tests.php (34 test cases) và Postman Collection thành viên 1. - Biên soạn Chương 1, Chương 2 (mục 2.1, 2.2) và Chương 3 (mục 3.1, 3.2) của báo cáo.	100% (Hoàn thành đúng hạn)	50%

MSSV	Họ và tên	Vai trò	Chi tiết công việc phụ trách (Mã nguồn & Báo cáo)	Mức độ hoàn thành	Tỷ lệ đóng góp
52400310	Nguyễn Anh Quân	Thành viên (Thành viên 2)	- Thiết kế lược đồ cơ sở dữ liệu quan hệ 3NF (database/schema.sql) và bộ dữ liệu thực nghiệm (database/seed.sql). - Xây dựng Front Controller (api/index.php, router.php, .htaccess) và các mô-đun tiện ích lõi (api/common/db.php, request.php, response.php). - Phát triển 3 phân hệ RESTful API: Categories (/api/categories/* - 5 endpoints), Books (/api/books/* - 5 endpoints kèm tìm kiếm, lọc, phân trang) và Statistics (/api/statistics/summary). - Xây dựng SDK kết nối API phía máy khách (public/assets/js/api.js) và 5 giao diện Frontend: Trang chủ (index.html), Chi tiết sách (book-detail.html), Bảng điều khiển thống kê (admin/dashboard.html), Quản trị sách (admin/books.html) và Quản trị thể loại (admin/categories.html). - Hợp nhất đặc tả docs/openapi.yaml, viết bộ kiểm thử tự động tests/run-member2-tests.php (31 test cases) và Postman Collection toàn hệ thống. - Biên soạn Chương 2 (mục 2.3), Chương 3 (mục 3.3), Chương 4, Chương 5 và Phụ lục của báo cáo.	100% (Hoàn thành đúng hạn)	50%

Phụ lục B: Thông tin truy cập hệ thống Demo, tài khoản kiểm thử và hướng dẫn vận hành nhanh

Để thuận tiện cho Giảng viên trong quá trình chấm điểm, đối chiếu mã nguồn và chạy kiểm thử thực nghiệm hệ thống ReadOwl Library, nhóm xin cung cấp đầy

đủ thông tin liên kết lưu trữ, danh sách tài khoản truy cập và quy trình thiết lập nhanh dưới đây:

Hạng mục thông tin	Giá trị / Đường dẫn / Thông số cấu hình chi tiết
Đường dẫn mã nguồn (Source Code Repository)	[CẦN BỔ SUNG: Dán đường dẫn GitHub / GitLab hoặc Google Drive chứa mã nguồn dự án của nhóm tại đây]
Đường dẫn Video thuyết trình & Demo	[CẦN BỔ SUNG: Dán đường dẫn YouTube / Google Drive chứa Video Demo dài 5-15 phút của nhóm tại đây]
Yêu cầu môi trường hệ thống	PHP >= 8.1 (khuyến nghị PHP 8.2, bật pdo_mysql, mbstring, openssl), MySQL >= 8.0 hoặc MariaDB >= 10.4, Composer 2.x
Tài khoản Quản trị viên (Admin)	Email: admin@example.com | Mật khẩu: Admin12345 (Toàn quyền quản trị kho sách, thể loại, độc giả, duyệt mượn trả và xem thống kê)
Tài khoản Độc giả chuẩn (Reader)	Email: an@example.com | Mật khẩu: ThuVien123 (Đang mượn 2 cuốn sách, còn hạn mức mượn thêm 1 cuốn)
Tài khoản Độc giả chạm hạn mức	Email: anh.hoang@example.com | Mật khẩu: ThuVien123 (Đang mượn đủ 3 cuốn sách, dùng để thử nghiệm chặn vượt hạn mức 409 Conflict)
Tài khoản Độc giả bị khóa	Email: bao.pham@example.com | Mật khẩu: ThuVien123 (Trạng thái is_active = 0, dùng để thử nghiệm chặn đăng nhập 403 Forbidden)
Tài liệu đặc tả OpenAPI 3.0.3	Tệp chính: docs/openapi.yaml (Có thể mở trực tiếp trên Swagger Editor tại https://editor.swagger.io hoặc nhập vào Postman)

Quy trình 5 lệnh cài đặt, khởi chạy máy chủ và thực thi bộ kiểm thử tự động 65 ca kiểm thử từ thư mục gốc của dự án:

```
# 1. Cài đặt thư viện phụ thuộc firebase/php-jwt qua Composer
composer install

# 2. Khởi tạo cơ sở dữ liệu library_rest và nạp dữ liệu mẫu vào
MySQL/MariaDB
mysql -u root -p < database/schema.sql
mysql -u root -p < database/seed.sql

# 3. Đồng bộ băm mật khẩu chuẩn Bcrypt cho các tài khoản mẫu và kiểm tra kết
nối PDO
php scripts/init-auth.php
```

```
# 4. Khởi chạy máy chủ web phát triển tại http://localhost:8000
php -S localhost:8000 router.php

# 5. Chạy 2 bộ kịch bản kiểm thử tích hợp tự động (65 test cases) ở cửa sổ
Terminal thứ hai
php tests/run-api-tests.php
php tests/run-member2-tests.php
```